



**UNIVERSIDADE
FEDERAL DO CARIRI**

Viewing 3D

Professora Dra. Luana Batista da Cruz
luana.batista@ufca.edu.br

Roteiro

01 Introdução

02 Viewing pipeline 3D

03 Parâmetros de coordenadas de visão 3D

04 Transformações do sistema de coordenadas de mundo para o de visão

05 Transformações de projeção

06 Funções OpenGL para viewing 3D

01

Introdução

Visão geral

Visualizando uma cena 3D

Projeções

Projeções - cenas realistas

Visão geral

- **Viewing pipeline 3D**

- As funções de viewing processam a descrição de objetos por meio de um conjunto de procedimentos a fim de projetar uma visão específica desses na superfície do dispositivo de saída

- Alguns desses procedimentos são similares aos do viewing pipeline 2D

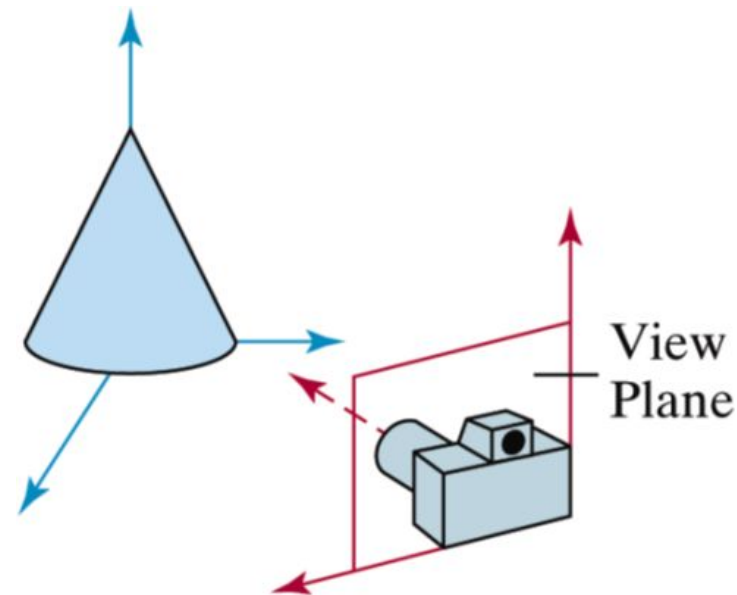
- Rotinas de recorte

- Mas outros são específicos do 3D

- Rotinas de projeção (ortogonal e perspectiva)
- Identificação de partes visíveis da cena
- Efeitos de luz

Visualizando uma cena 3D

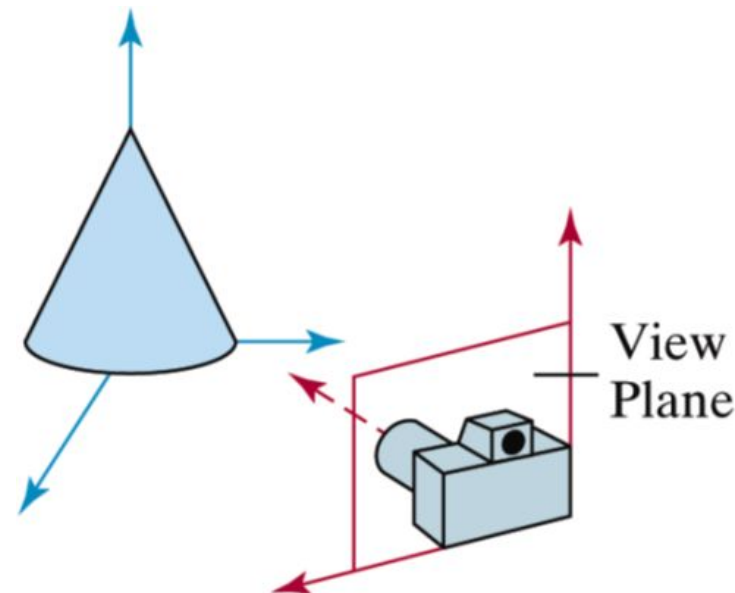
- Para obter a visão de uma cena 3D descrita por coordenadas no sistema do mundo, é necessário, primeiramente, definir um sistema de referência da câmera (ou sistema de visão). Isso envolve
 - Definir a posição, orientação e direção da câmera no espaço
 - Estabelecer o plano de visão (ou plano de projeção)



Visualizando uma cena 3D

- Para obter a visão de uma cena 3D descrita por coordenadas no sistema do mundo, é necessário, primeiramente, definir um sistema de referência da câmera (ou sistema de visão). Isso envolve
 - Definir a posição, orientação e direção da câmera no espaço
 - Estabelecer o plano de visão (ou plano de projeção)

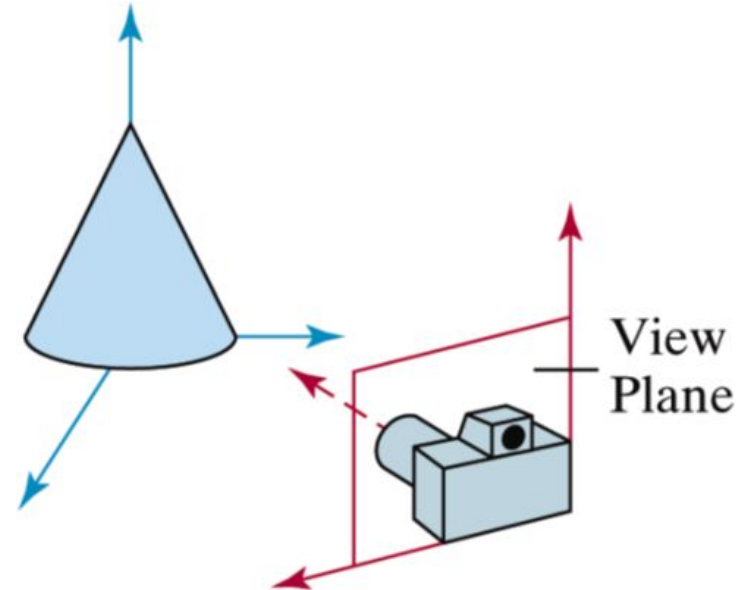
A **posição** da câmera fica na origem do sistema de coordenadas de visão



Visualizando uma cena 3D

- Para obter a visão de uma cena 3D descrita por coordenadas no sistema do mundo, é necessário, primeiramente, definir um sistema de referência da câmera (ou sistema de visão). Isso envolve
 - Definir a posição, orientação e direção da câmera no espaço
 - Estabelecer o plano de visão (ou plano de projeção)

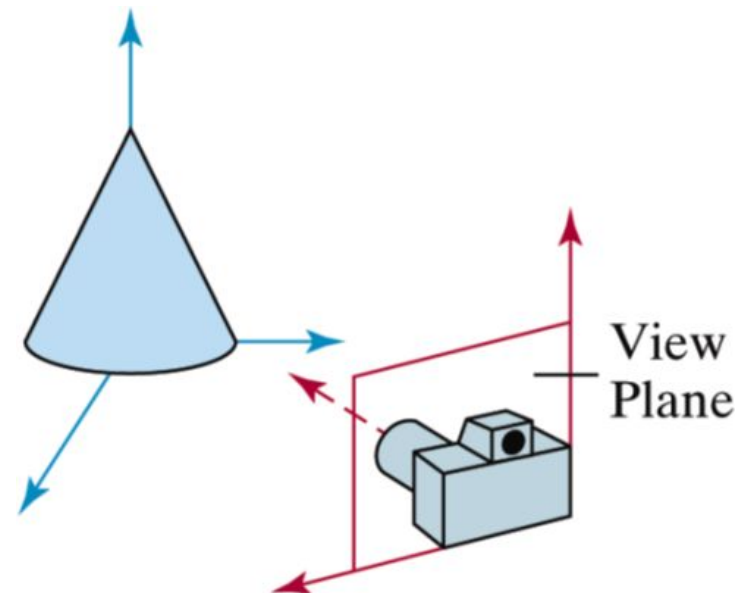
A **orientação** determina como a câmera está rotacionada em relação aos eixos do mundo



Visualizando uma cena 3D

- Para obter a visão de uma cena 3D descrita por coordenadas no sistema do mundo, é necessário, primeiramente, definir um sistema de referência da câmera (ou sistema de visão). Isso envolve
 - Definir a posição, orientação e direção da câmera no espaço
 - Estabelecer o plano de visão (ou plano de projeção)

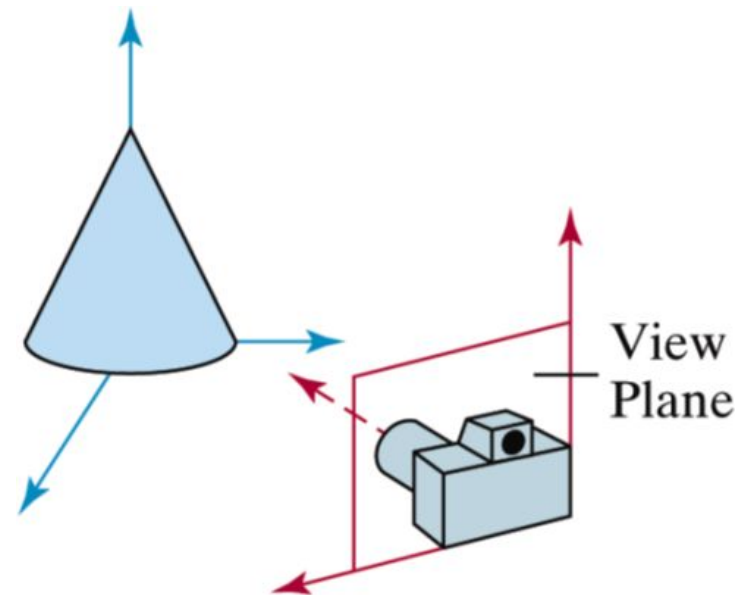
A **direção** de visão da câmera é para onde a câmera está apontando



Visualizando uma cena 3D

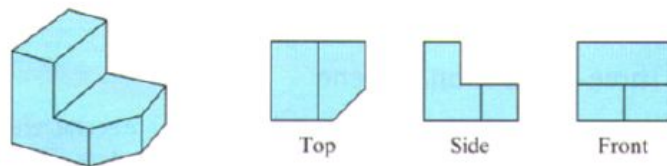
- Para obter a visão de uma cena 3D descrita por coordenadas no sistema do mundo, é necessário, primeiramente, definir um sistema de referência da câmera (ou sistema de visão). Isso envolve
 - Definir a posição, orientação e direção da câmera no espaço
 - Estabelecer o plano de visão (ou plano de projeção)

O **plano de visão** é como se fosse o filme da câmera, onde os objetos serão projetados



Projeções

- É possível escolher diferentes métodos para projetar uma cena no plano de visão
 - **Projeção paralela:** projeta os pontos de um objeto ao longo de linhas paralelas (não altera o tamanho). Usado para desenhos arquitetônicos e de engenharia
 - **Projeção perspectiva:** projeta os pontos de um objeto ao longo de caminhos convergentes. Cria cenas mais realistas (objetos longe da posição de visão são mostrados menores)



(a) Projeção Paralela



(b) Projeção Perspectiva

Projeções

- A **informação de profundidade é importante** para a composição de uma cena 3D, pois ajuda a identificar a parte da frente e de trás dos objetos



(a)



(b)



(c)

Problemas de ambiguidade pela falta de informação de profundidade em (a), que pode interpretar como (b) ou (c)

Projeções - cenas realistas

- Para cenas realistas, as partes não visíveis de um objeto são completamente eliminadas, e somente as visíveis são mostradas
- Os pixels da tela terão informação apenas das cores das superfícies da frente

Projeções - cenas realistas

- Efeitos realistas são alcançados usando **efeitos de iluminação**, que são aplicados depois da projeção
 - Define-se a luz ambiente
 - Especifica-se a localização e cor das diferentes fontes de luz
- As **características dos materiais** são também especificadas
 - Transparente, opaco, rugoso, brilhante, cor, etc



02

Viewing pipeline 3D

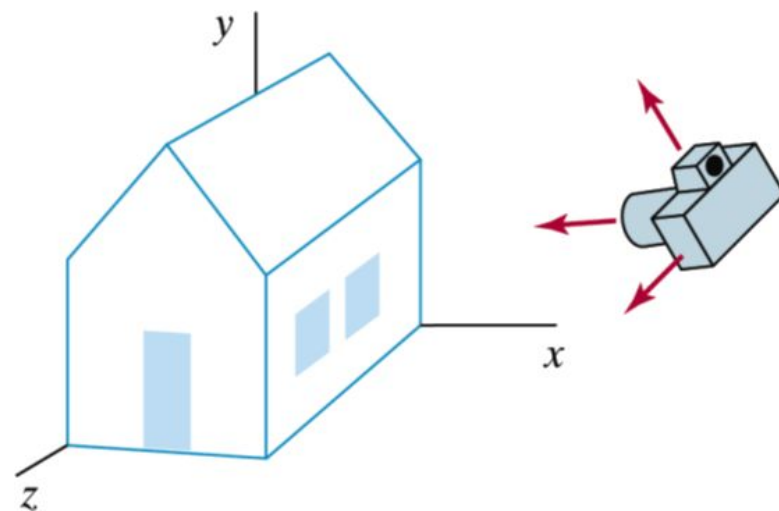
Visão geral

Viewing pipeline 3D

- O **sistema de coordenadas de visão** (ou sistema de coordenadas da câmera) é um sistema de referência alinhado à câmera virtual
- Para construí-lo, são utilizados os parâmetros da câmera definidos com base no sistema de coordenadas do mundo. A partir desses parâmetros, o sistema de visão é formado, determinando como os objetos serão observados da perspectiva da câmera
- Lembrando que o sistema criado deve ser **ortonormal**, ou seja, composto por vetores que são ortogonais (perpendiculares entre si) e unitários (de comprimento igual a 1)

Viewing pipeline 3D

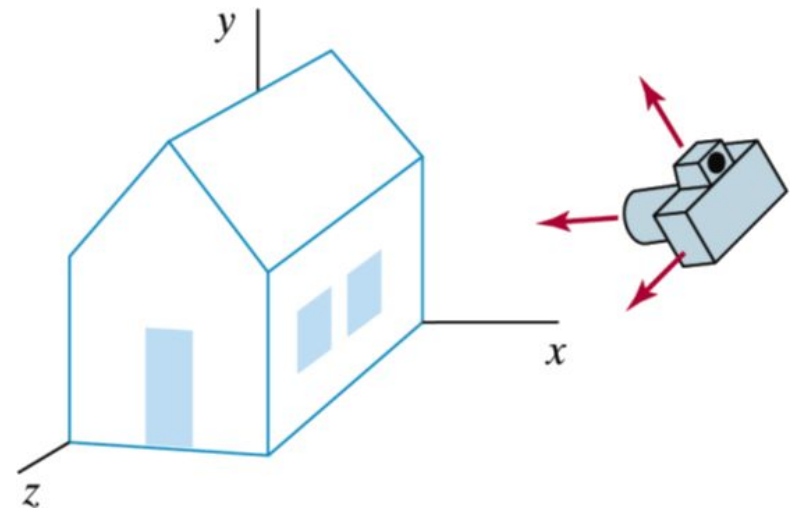
- Para construir o sistema de visão, é necessário definir alguns parâmetros da câmera, semelhante a tirar uma fotografia
 - **Posição da visão:** onde a câmera será colocada
 - **Orientação da câmera:** como ela estará rotacionada, definindo a posição up (eixo y_{view})
 - **Direção da câmera:** para onde ela está apontando (eixo z_{view})



Viewing pipeline 3D

- Para construir o sistema de visão, é necessário definir alguns parâmetros da câmera, semelhante a tirar uma fotografia
 - **Posição da visão:** onde a câmera será colocada
 - **Orientação da câmera:** como ela estará rotacionada, definindo a posição up (eixo y_{view})
 - **Direção da câmera:** para onde ela está apontando (eixo z_{view})

Após configurado, é possível projetar a cena no plano de visão e gerar a imagem 3D

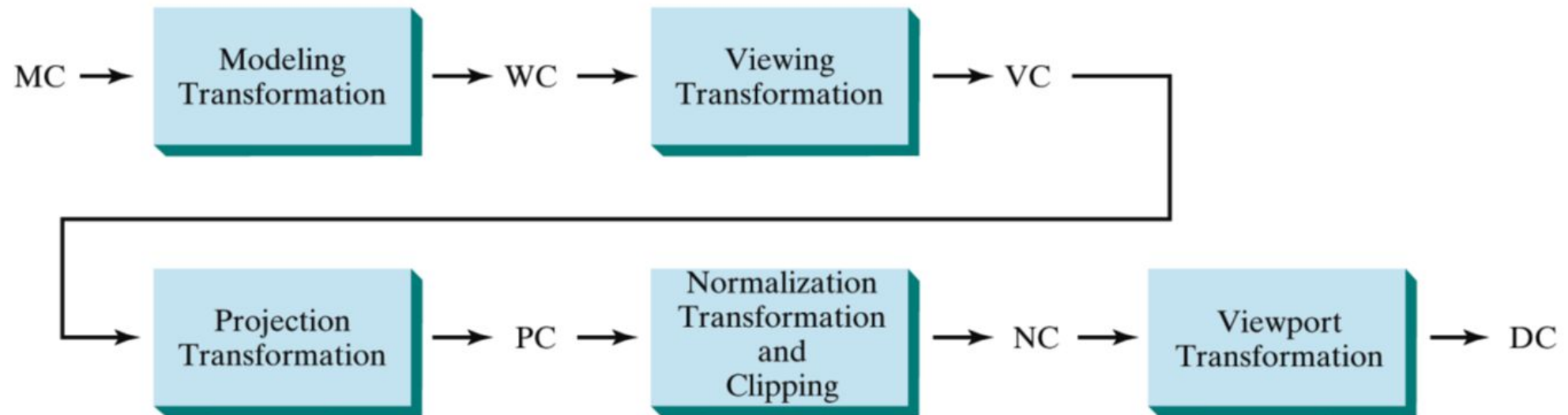


Viewing pipeline 3D

- Algumas das operações do viewing pipeline 3D são semelhantes ao 2D
 - Uma **viewport 2D** é usada para posicionar a visão projetada no dispositivo de saída. O que for projetado no plano de visão vai ser passado para a viewport
 - Uma **janela de recorte 2D** é usada para selecionar uma visão que será mapeada na viewport. Posicionada no plano de visão
 - Uma janela de saída (OpenGL) é definida em coordenadas da tela (pixels)
- Porém, outras são diferentes
 - A janela de recorte é posicionada sobre um plano de visão, e a cena é recortada considerando um volume no espaço (**volume de recorte**) usando planos de recorte
- É importante ressaltar que a **posição de câmera**, o **plano de visão**, a **janela de recorte** e os **planos de recorte** são especificados dentro do sistema de coordenadas de mundo

Viewing pipeline 3D

- Mapeamento de uma cena 3D no sistema de coordenadas do mundo para o sistema de coordenadas do dispositivo

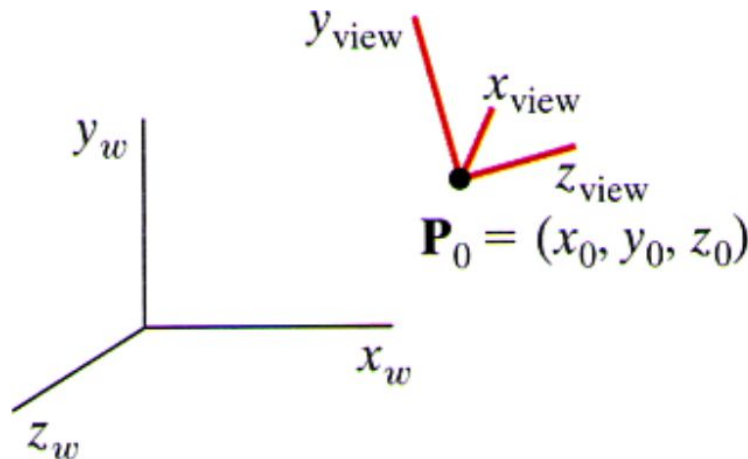


Parâmetros de coordenadas de visão 3D

Vetor normal ao plano de visão
Sistema de coordenada de visão uvn
Gerando efeitos de visão 3D

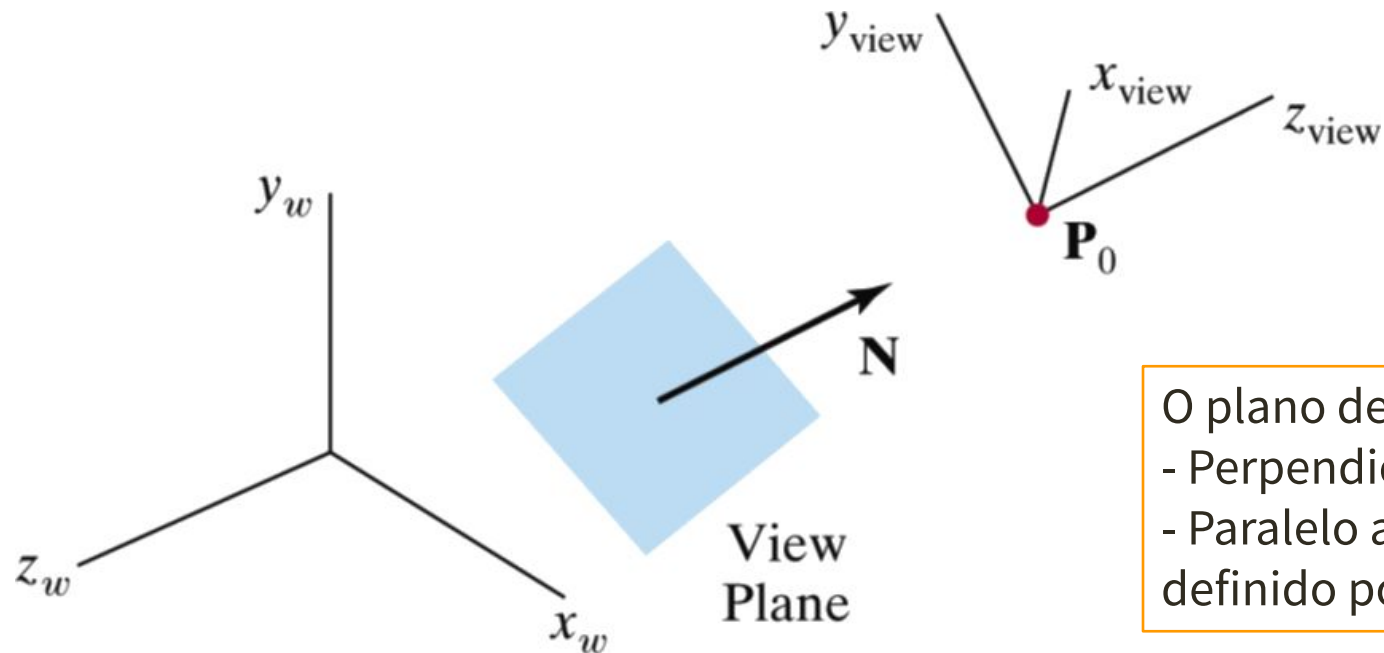
Parâmetros de coordenadas de visão 3D

- Estabelecendo um sistema de coordenadas de visão 3D
 - A **origem do sistema** $P_0 = (x_0, y_0, z_0)$, chamada de **ponto de visão** (posição da câmera ou do olho)
 - O **vetor view-up** V , que define a direção de y_{view} , e fornece a orientação da câmera
 - Uma segunda direção para um dos eixos coordenados restantes, normalmente o z_{view} , com a **direção de visão** ao longo desse eixo



Vetor normal ao plano de visão

- Como a direção de visão em geral é definida sobre o eixo z_{view} , o **plano de visão** é normalmente assumido ser perpendicular a esse eixo
- Assim, a **orientação do plano de visão** e a **direção positiva do eixo z_{view}** podem ser definidas com um **vetor normal N** ao plano de visão

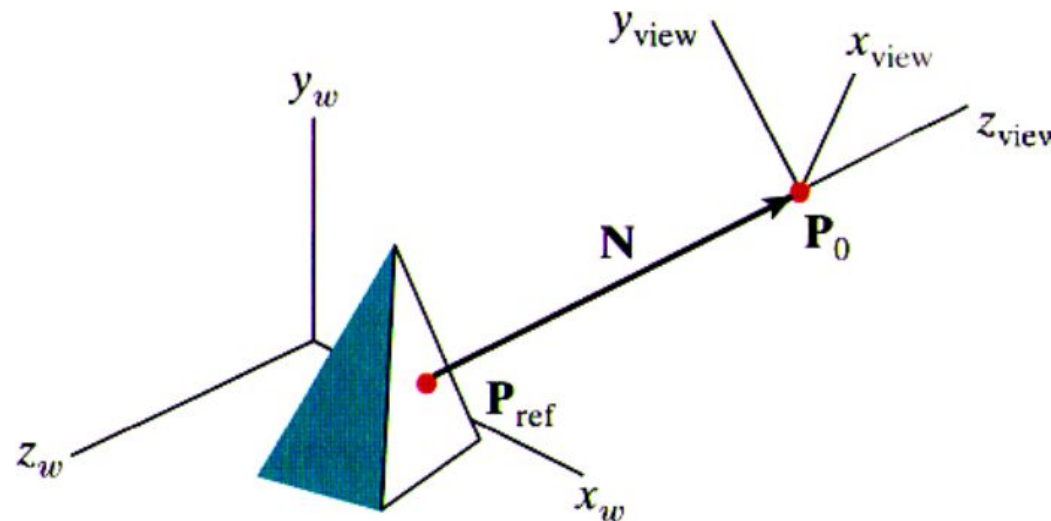


O plano de visão é

- Perpendicular a z_{view} ou
- Paralelo ao plano definido por x_{view} e y_{view}

Vetor normal ao plano de visão

- O **vetor normal N** e $\mathbf{z}_{\text{view}}$ podem ser especificados de várias formas
 - A forma mais simples é por meio de uma linha partindo de um ponto de referência P_{ref} até a origem de visão P_0
 - Nesse caso, esse ponto de referência é denominado **look-at point**, com a direção de visão oposta a de N ($\mathbf{z}_{\text{view}}$)

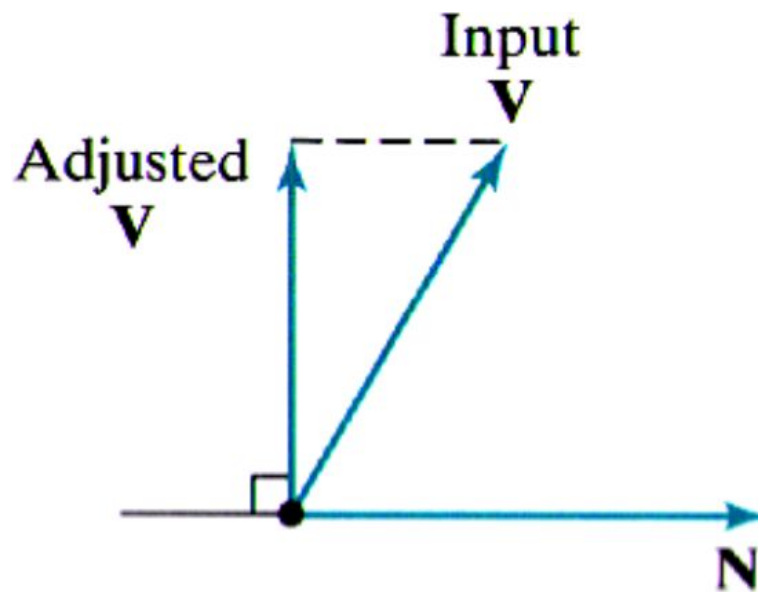


Vetor normal ao plano de visão

- Uma vez estabelecida a direção normal ao plano de visão N , podemos estabelecer o **vetor view-up V** que dá o **eixo y_{view}**
 - Usualmente V é definido selecionando uma posição relativa a origem do sistema de coordenadas do mundo

Vetor normal ao plano de visão

- V pode ser definido em qualquer direção, desde que não paralelo a N
 - Uma forma conveniente seria definir uma direção paralela ao eixo y_w , por exemplo, $V = (0, 1, 0)$
 - Se esse não for exatamente perpendicular a N, as rotinas de visão podem ajustá-lo projetando-o em um plano perpendicular a N



Ajuste do vetor
view-up V para
torná-lo
perpendicular a N

Sistema de coordenadas de visão uvn

- Como a normal N define a orientação z_{view} , e o vetor view-up V é usado para definir y_{view} , só é necessário definir a direção positiva de x_{view}
 - A direção x_{view} é representada pelo vetor U , calculada pelo produto vetorial de V e N
 - O produto vetorial de N e U também pode ser usado para ajustar o valor de V ao longo do eixo y_{view}

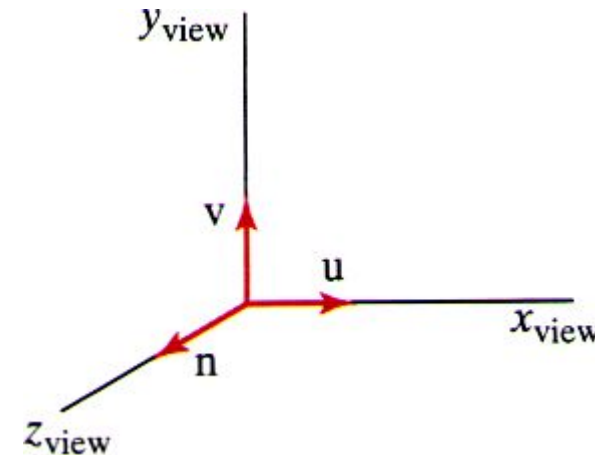
Sistema de coordenadas de visão uvn

- Para se obter o sistema de coordenadas uvn fazemos

$$n = \frac{N}{|N|} = (n_x, n_y, n_z)$$

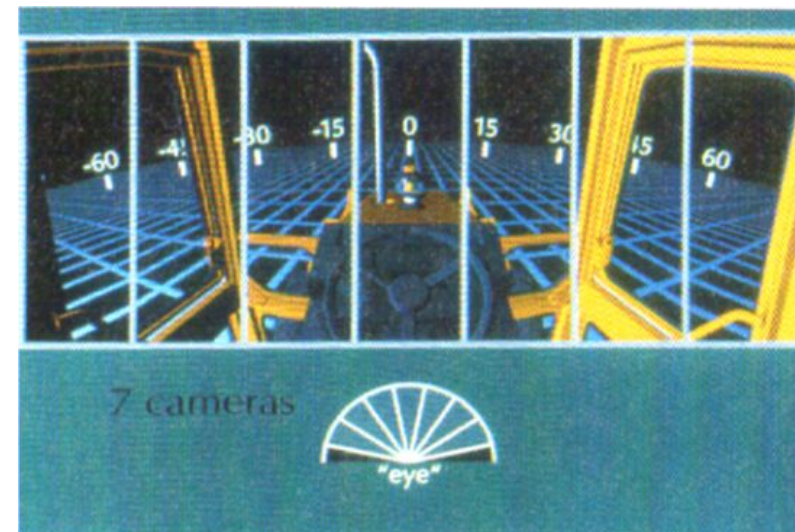
$$u = \frac{V \times n}{|V \times n|} = (u_x, u_y, u_z)$$

$$v = n \times u = (v_x, v_y, v_z)$$



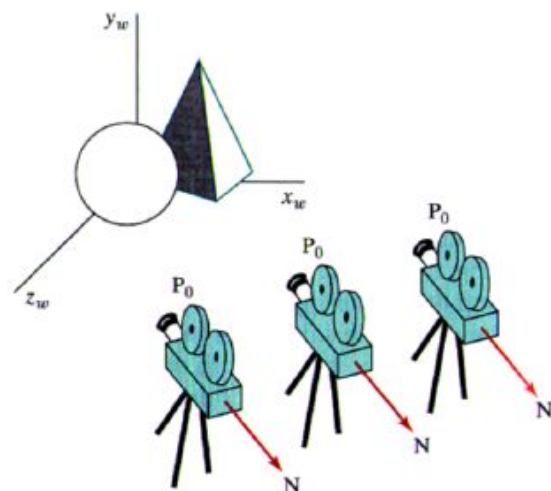
Gerando efeitos de visão 3D

- Variando-se os parâmetros de visão é possível obter diferentes efeitos
 - De uma **posição de visão fixa** é possível mudar N para mostrar objetos em posições ao redor da origem (alterando o N e o x_{view})
 - Variar N para criar uma cena composta de múltiplas visões de uma posição fixa da câmera
 - Quando alterar N, não se esqueça de mudar os outros eixos para manter o sistema orientado e ortonormal

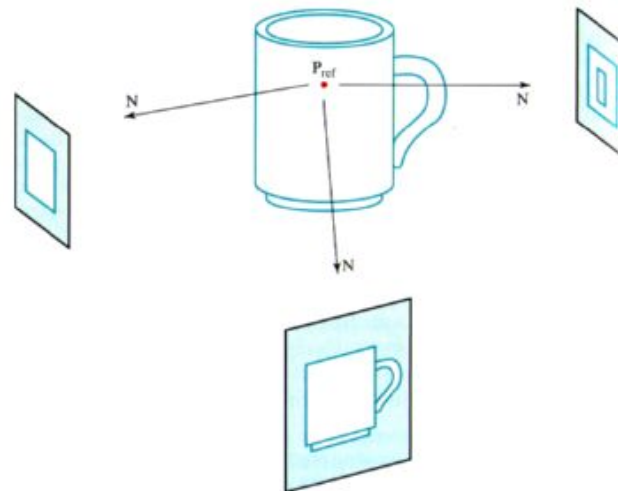


Gerando efeitos de visão 3D

- Efeitos de mover a câmera (pam) podem ser obtidos fixando N (direção) e a orientação e modificando o ponto de visão (posição da câmera)
- Para mostrar diferentes visões de uma objeto (visão lateral, frontal, etc.) podemos mover o ponto de visão ao redor do objeto (muda junto a orientação, direção e posição)



(a) Efeito de pam



(b) Visões diferentes

Transformação do sistema de coordenadas de mundo para o de visão

Transformação do sistema de coordenadas de mundo para o de visão

- No viewing pipeline 3D, o primeiro passo a ser executado após a cena ser construída é transferir a descrição dos objetos ao sistema de coordenadas de visão
 - Essa conversão é equivalente a sobrepor o sistema de referência de visão sobre o sistema de referência de mundo
- Esse mapeamento pode ser feito em dois passos
 - 1) Transladando a origem do sistema de coordenadas de visão para a origem do sistema de coordenadas de mundo
 - 2) Rotacionando para alinhar os eixos x_{view} , y_{view} e z_{view} , com os eixos de mundo x_w , y_w e z_w

Transformação do sistema de coordenadas de mundo para o de visão

- Se a origem do sistema de visão for $P_0 = (x_0, y_0, z_0)$ a matriz de translação será

$$T = \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Transformação do sistema de coordenadas de mundo para o de visão

- A matriz de rotação pode ser obtida direto dos vetores $u = (u_x, u_y, u_z)$, $v = (v_x, v_y, v_z)$ e $n = (n_x, n_y, n_z)$
- Só é possível montar essa matriz única porque é um sistema ortonormal

$$R = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

u é a versão normalizada de x_{view}
 v é a versão normalizada de y_{view}
 n é a versão normalizada de z_{view}

Transformação do sistema de coordenadas de mundo para o de visão

- A matriz de transformação é portanto

$$M_{WC,VC} = R \cdot T = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Transformações de projeção

Projeções ortogonais

Projeções perspectivas

Transformações de viewport e coordenadas de tela 3D

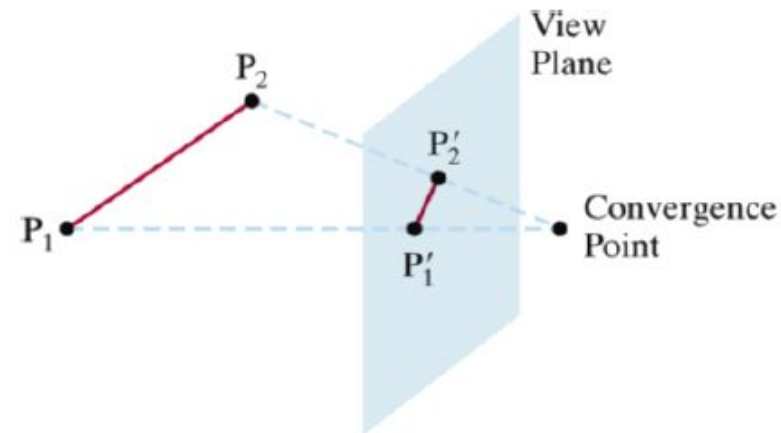
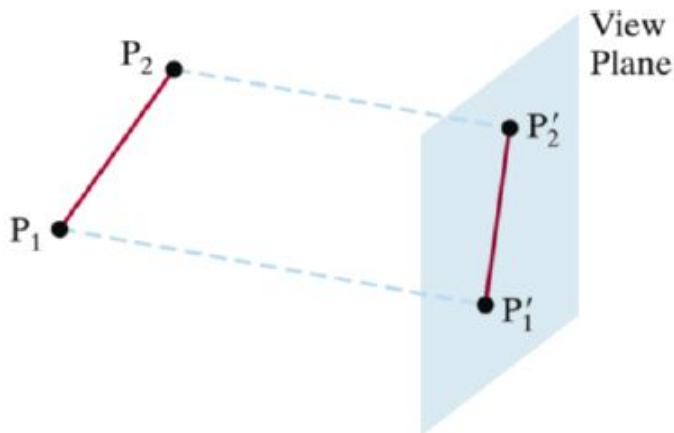
Transformações de projeção

- **Relembrando**

- A câmera possui uma posição definida no sistema de coordenadas do mundo, por exemplo $(3, 3, 3)$
- Os objetos também têm suas posições nesse mesmo sistema
- Ao transformar os objetos para o sistema de coordenadas da câmera (visão), cada objeto passa a ter uma nova posição relativa à câmera
- Nesse novo sistema, a origem da câmera em relação a si mesma é $(0, 0, 0)$, e todas as posições dos objetos são medidas em relação a essa origem

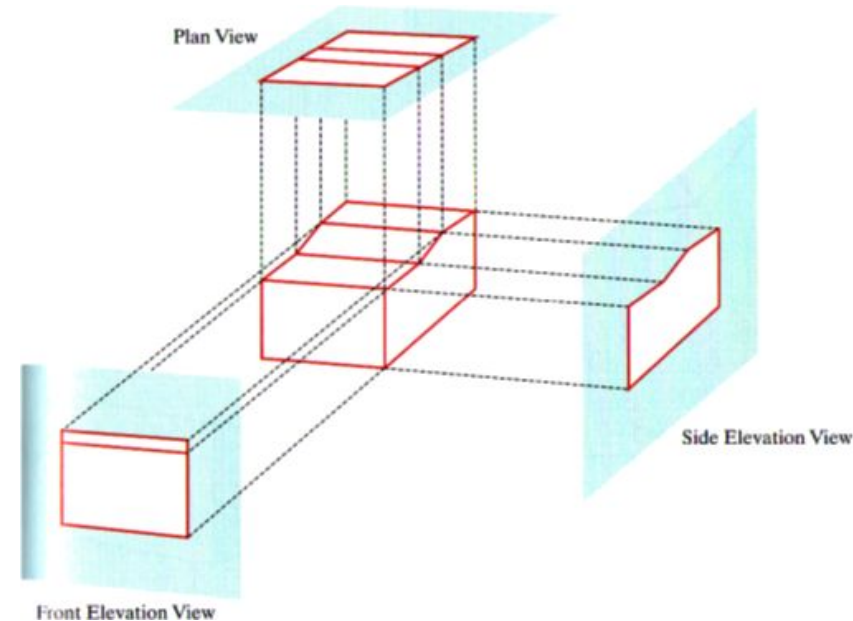
Transformações de projeção

- Após a transformação para as coordenadas de visão, o próximo passo do viewing pipeline 3D é a projeção no plano de projeção/visão
- Em geral os pacotes gráficos suportam
 - **Projeção paralela:** as coordenadas são transferidas para o plano de projeção ao longo de linhas paralelas ao vetor N
 - **Projeção perspectiva:** as coordenadas são transferidas para o plano de projeção ao longo de linhas que convergem a um ponto



Projeções ortogonais

- Transformação da descrição dos objetos a um plano de projeção ao longo de linhas paralelas ao vetor normal N do plano de projeção
- Frequentemente usada para produzir a visão frontal, lateral e superior de um objeto. Lembrando que o plano de visão acompanha a câmera
- Preserva tamanhos e ângulos: útil para desenhos arquitetônicos e de engenharia



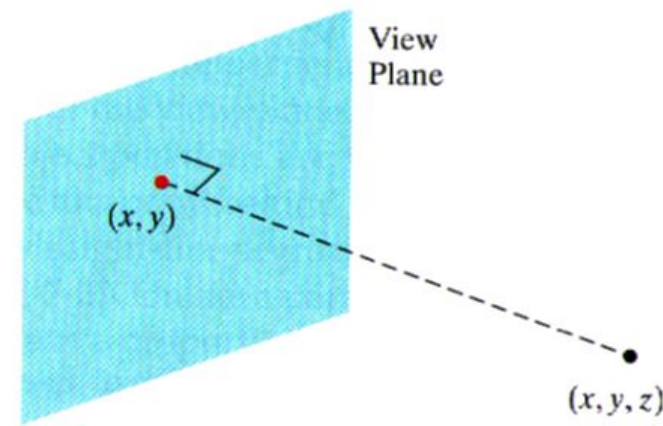
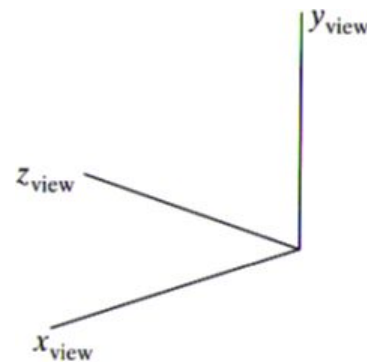
Coordenadas de projeções ortogonais

- Com a direção de projeção paralela a z_{view} , as equações para a transformação de projeção ortogonal de um posição (x, y, z) são

$$x_p = x$$

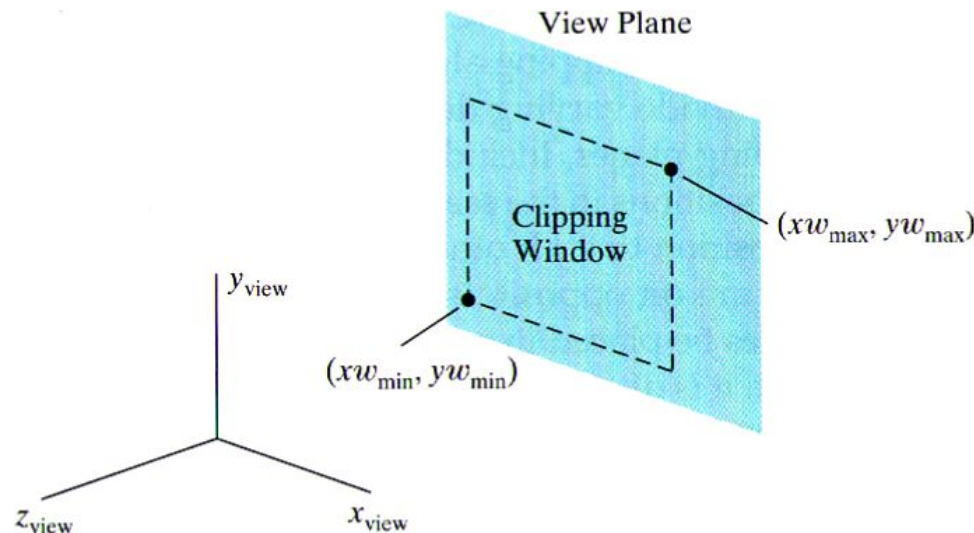
$$y_p = y$$

- O valor de z é armazenado para uso futuro nos procedimentos para determinar visibilidade



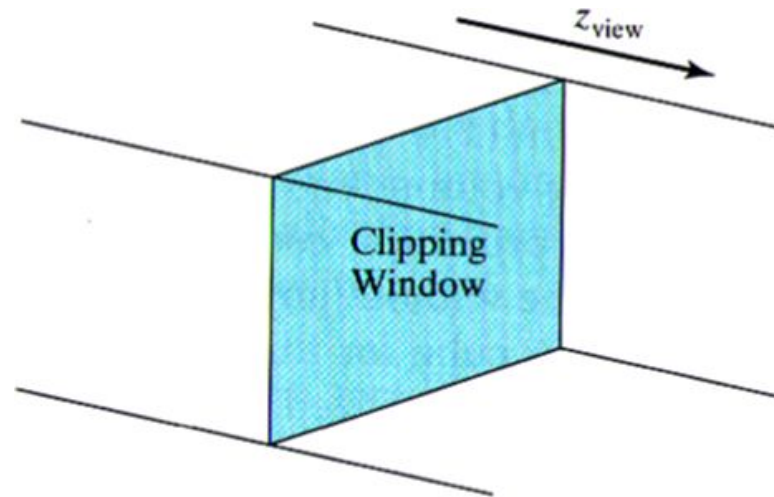
Janela de recorte e volume de projeção ortogonal

- O plano de visão é infinito, mas a tela dos dispositivos são finitos. Também é preciso delimitar a porção do plano de visão que vai ser jogada na tela
- Para se determinar o quanto de uma cena 3D será transferida para o plano de projeção, uma **janela de recorte** é utilizada
 - É necessário determinar os limites dessa janela sobre o plano de projeção com as arestas paralelas aos eixos x_{view} e y_{view}

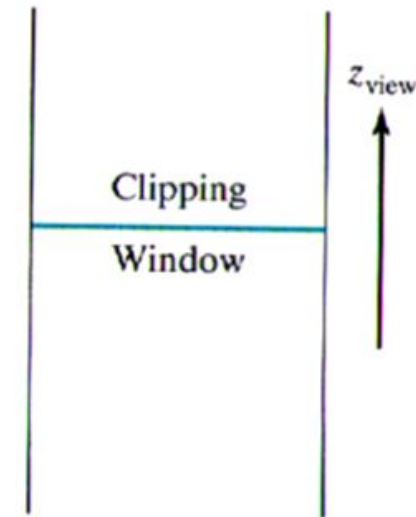


Janela de recorte e volume de projeção ortogonal

- As arestas de janela de recorte especificam os limites x e y da parte da cena que será mostrada, formando o **volume de visão de projeção ortogonal**



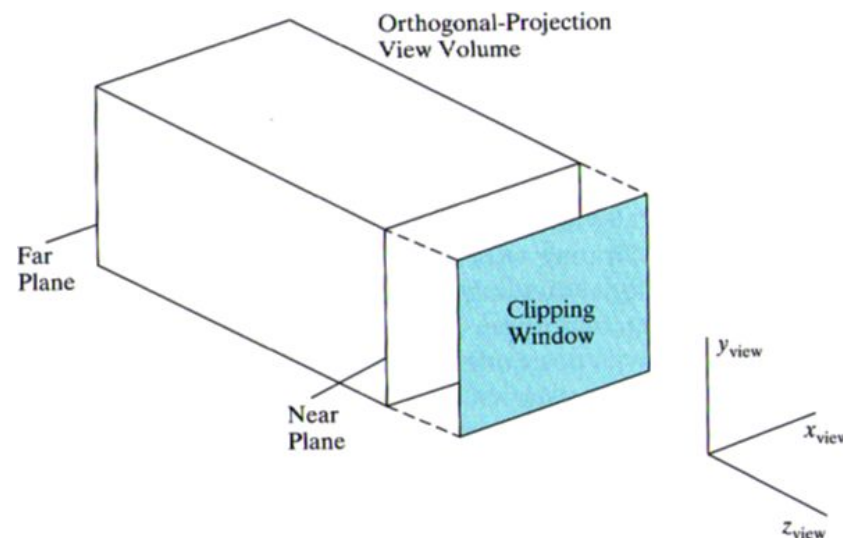
Side View
(a)



Top View
(b)

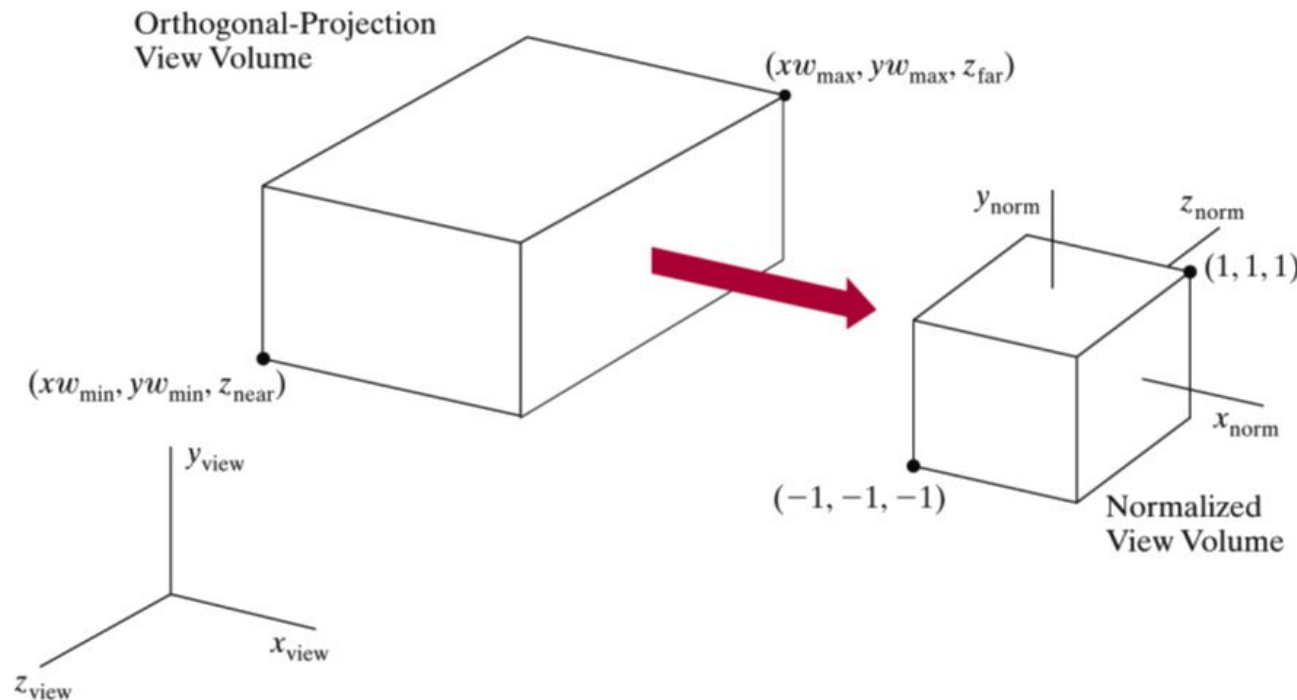
Janela de recorte e volume de projeção ortogonal

- Para se limitar a extensão desse volume na direção z_{view} , dois planos de fronteira paralelos aos planos de visão, chamados **planos de recorte near/far**, são considerados
 - Permite eliminar objetos que estão na frente ou atrás de uma parte da cena
 - Com a direção de visão ao longo do eixo negativo z_{view} , logo $z_{\text{far}} < z_{\text{near}}$
 - Ao definir o near, estou delimitando a posição do plano de visão (janela de recorte)



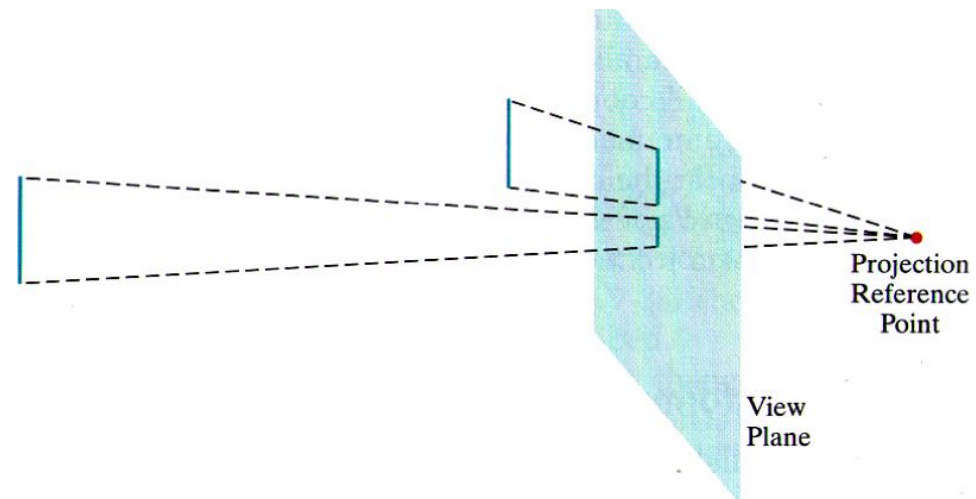
Transformação de normalização para projeção ortogonal

- Como qualquer posição (x, y, z) em uma projeção ortogonal é mapeada para (x, y) , as coordenadas dentro do volume de visão são as coordenadas de projeção, assim essas podem ser mapeadas para um volume de visão normalizado sem precisar ser reprojeta



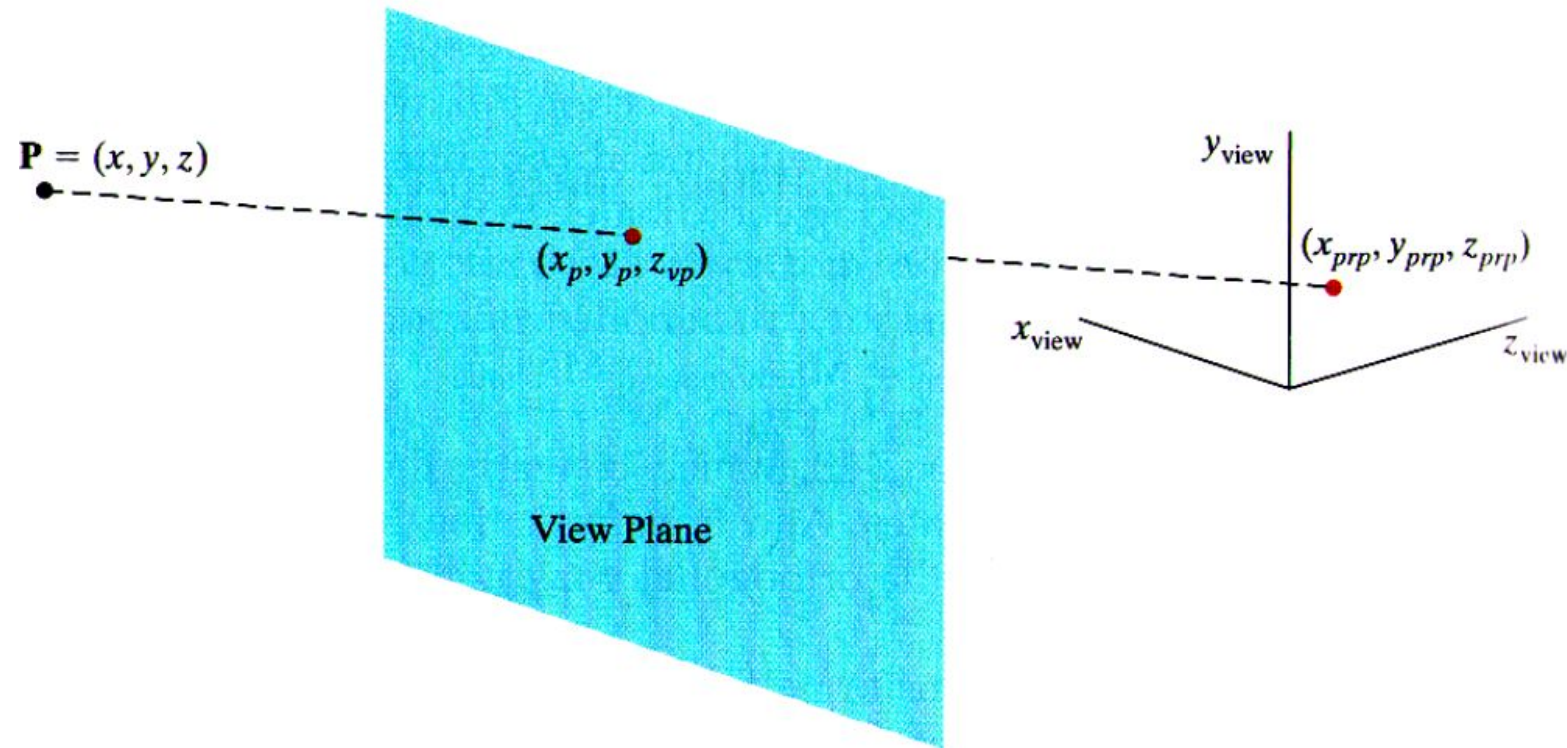
Projeções perspectivas

- Para conseguir maior realismo que o obtido nas projeções paralelas temos que considerar que os raios de luz refletidos na cena seguem caminhos convergentes
- Isso pode ser aproximado projetando objetos ao plano de visão ao longo de caminhos convergentes a uma posição chamada **ponto de referência de projeção** (ou **ponto de convergência**)



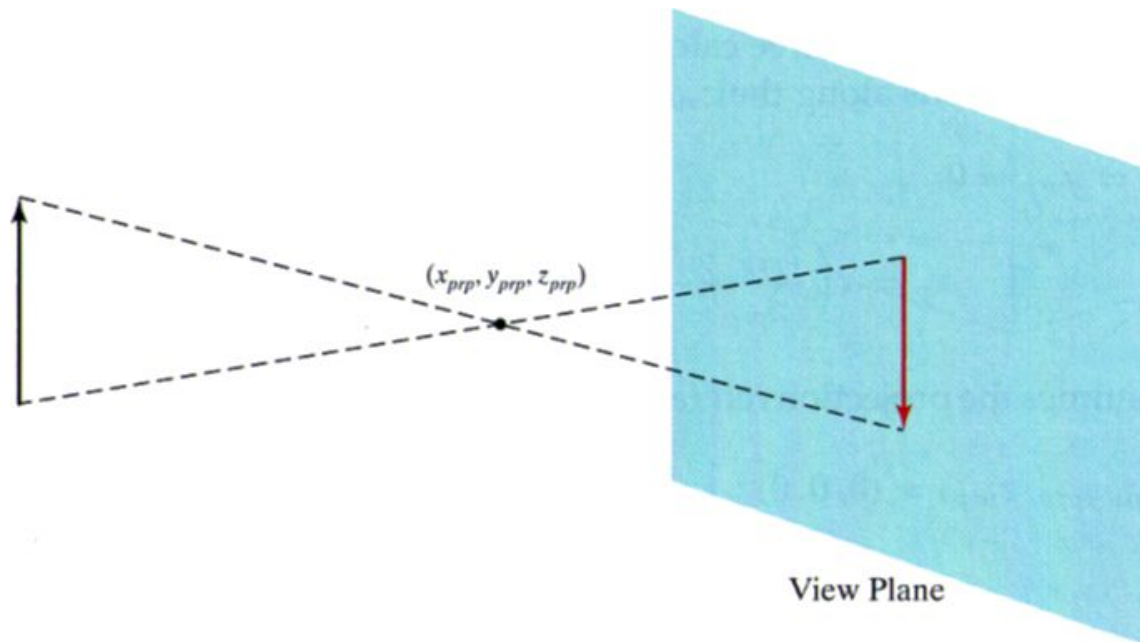
Transformação de coordenadas de projeção perspectiva

- Algumas bibliotecas gráficas permitem que se escolha o ponto de referência de projeção $(x_{prp}, y_{prp}, z_{prp})$



Casos especiais das equações de projeção perspectiva

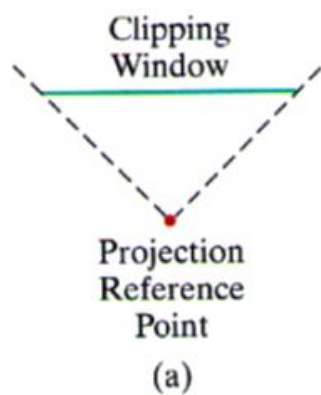
- Em geral, o plano de projeção está entre o ponto de referência de projeção e a cena, mas outras posições são possíveis (menos ambos coincidindo)



Os objetos são invertidos se o ponto de referência de projeção está entre o plano de visão e a cena (objetos)

Casos especiais das equações de projeção perspectiva

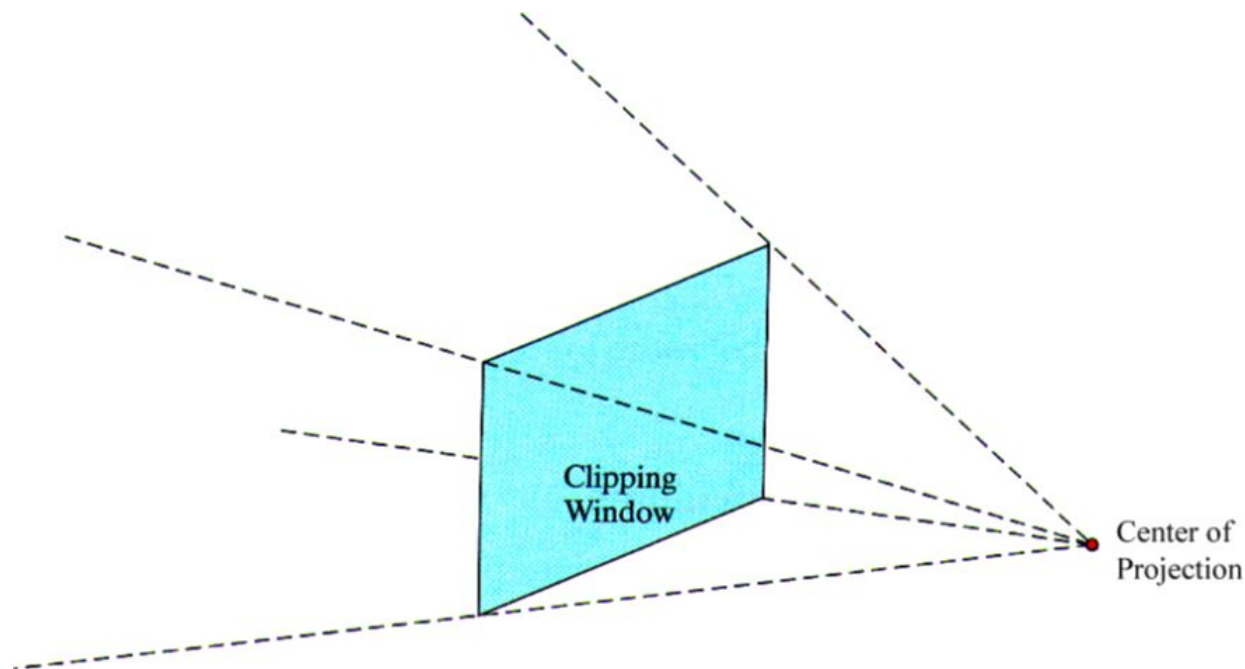
- Os efeitos de perspectiva também dependem da distância entre o ponto de referência de projeção e o plano de visão



Quando o ponto de referência de projeção está próximo ao plano de visão, os efeitos da perspectiva são enfatizados (objetos mais próximos ao plano aparecerão muito maiores do que os distantes)

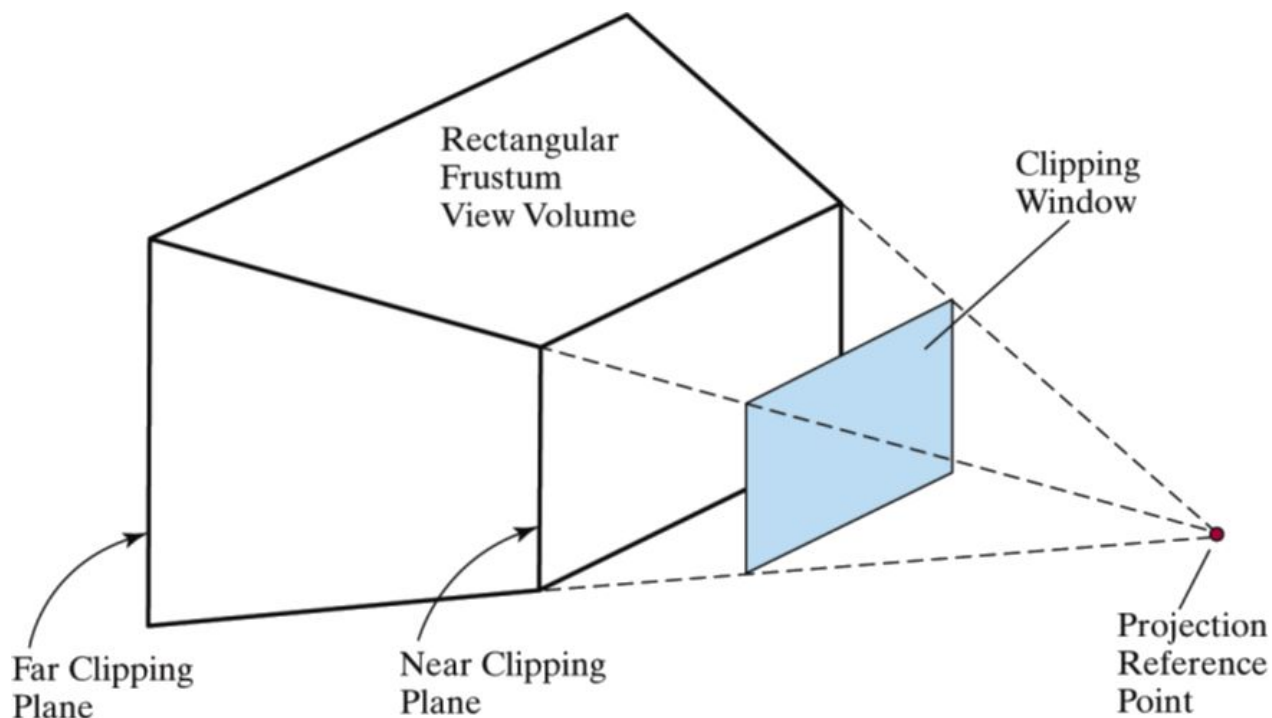
Volume de projeção perspectiva

- Em uma projeção perspectiva, o volume de visão definido é uma pirâmide infinita com seu ápice no centro de projeção, normalmente chamada de **pirâmide de visão**
 - Objetos fora dessa pirâmide são eliminados pelas rotinas de recorte



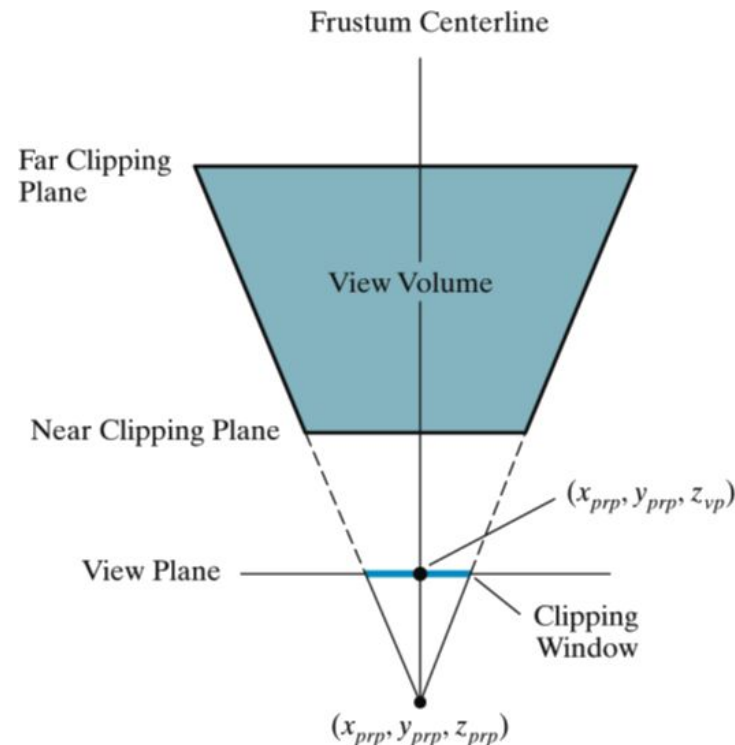
Volume de projeção perspectiva

- Adicionando os planos de recorte near/far perpendiculares ao eixo z_{view} essa pirâmide é truncada resultando em um tronco de pirâmide (**frustum**)



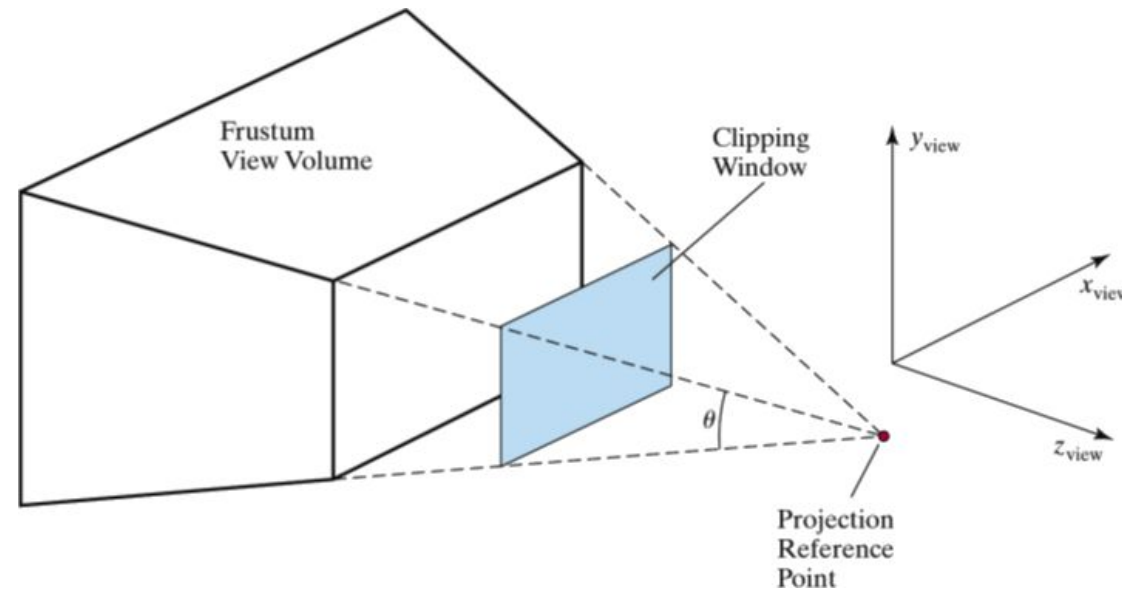
Frustum simétrico de projeção perspectiva

- A linha que sai do ponto de referência de projeção e atravessa a janela de recorte (plano de visão) e volume de visão é a linha central do frustum
 - Se essa for perpendicular ao plano de visão, temos um **frustum simétrico**



Frustum simétrico de projeção perspectiva

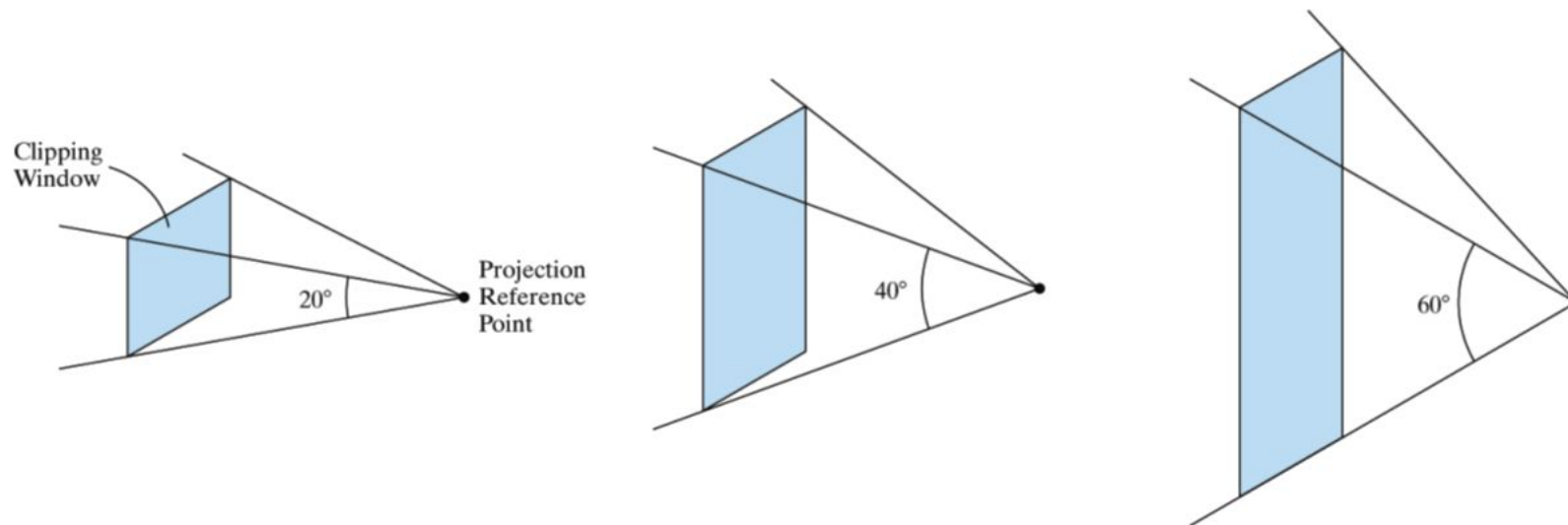
- Uma projeção perspectiva pode também ser aproximada considerando o **cone de visão**, definido pelo **ângulo do campo de visão**, de uma câmera
 - Grandes ângulos de campo de visão correspondem a lentes grandes-angulares



- De forma geral, o ângulo do campo de visão é definido entre o plano de recorte superior e o inferior do frustum

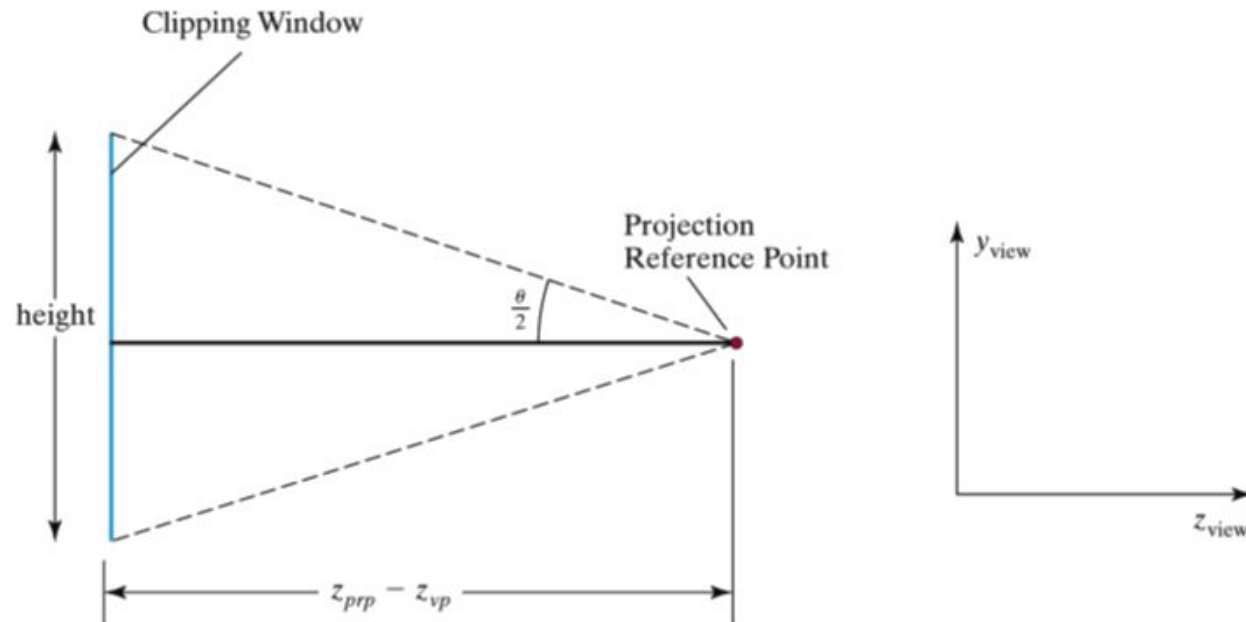
Frustum simétrico de projeção perspectiva

- Diminuir o ângulo do campo de visão diminui a janela de recorte
- Aumentar o ângulo do campo de visão aumenta a janela de recorte



Frustum simétrico de projeção perspectiva

- Dado um ponto de referência e a posição do plano de visão, o **ângulo do campo de visão** determina a altura da janela de recorte

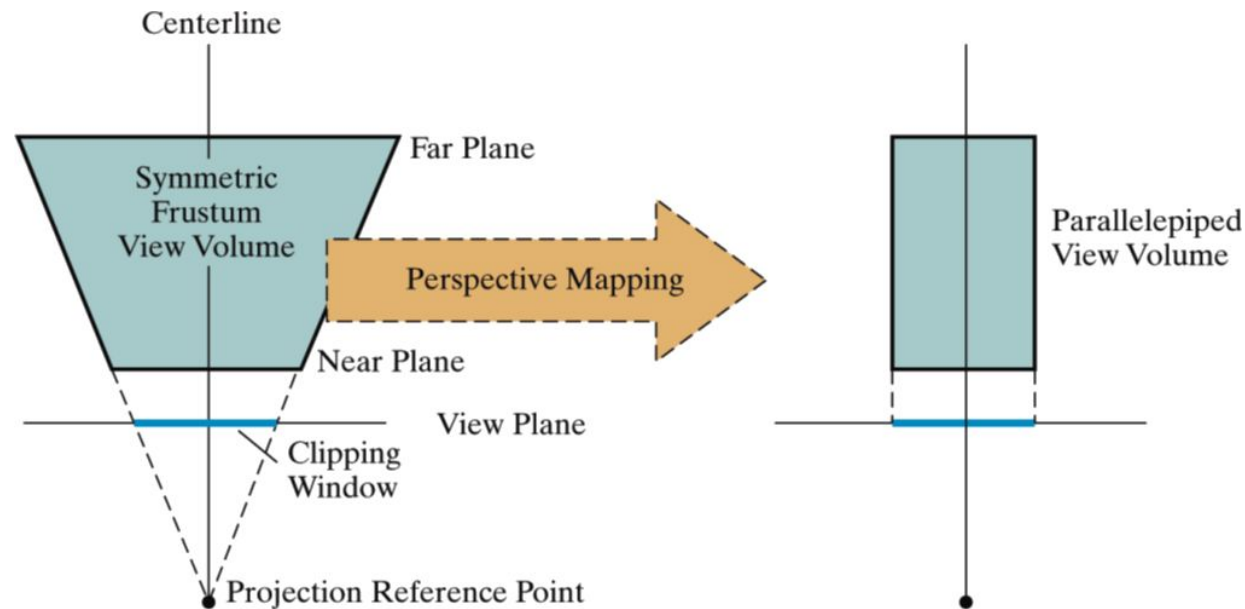


- Para definir a largura é necessário considerar um parâmetro adicional que pode ser a **razão de aspecto** $aspect = (width/height)$

Frustum simétrico de projeção perspectiva

- **Observação importante!**

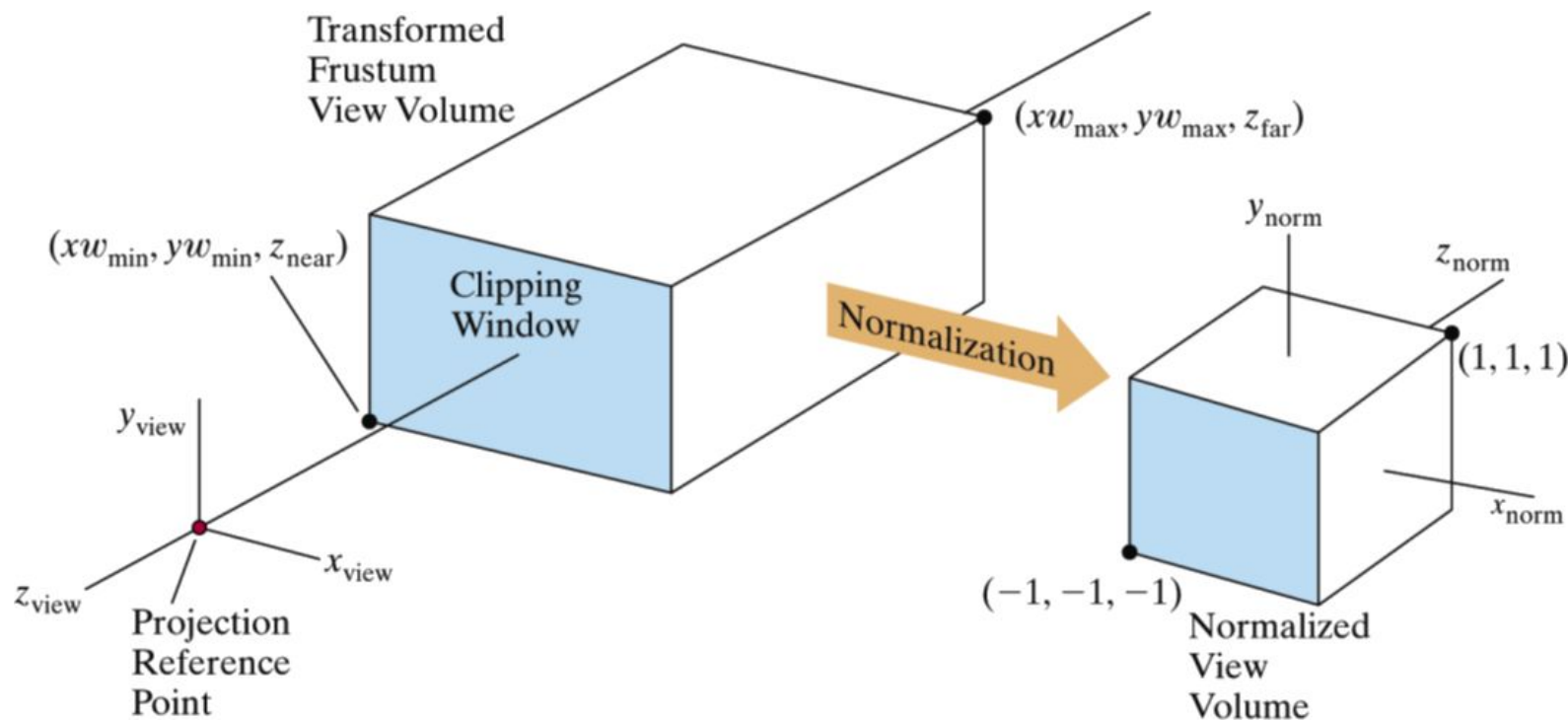
- Para facilitar a renderização da cena, as coordenadas de um ponto no frustum 3D (pirâmide) são convertidas para um sistema de coordenadas mais simples e regular (paralelepípedo)



Após a projeção perspectiva, as coordenadas dos objetos estão no intervalo da janela de recorte

Transformação de projeção perspectiva normalizada

- O último passo da projeção perspectiva é mapear o paralelepípedo obtido para um **volume de visão normalizado**



Transformação de viewport e coordenadas de tela 3D

- Após o conteúdo do volume de visão ter sido definido, esse pode ser transferido para coordenadas da tela (antes passa para uma etapa intermediária, a viewport)
- Esse é um processo semelhante ao 2D, porém informação de profundidade é preservada para teste de visibilidade e rendering
- As posições x e y são enviadas para o **frame buffer** (informação de cor para os pontos na tela)
- Os valores de z são enviados para o **depth buffer** para serem usados em rotinas para determinação de cor e visibilidade

06

Funções OpenGL para viewing 3D

Função de projeção ortogonal e perspectiva simétrica

Funções OpenGL para viewing 3D

- A biblioteca **OpenGL** inclui funções para
 - Transformação do sistema de coordenadas de mundo para o de visão
 - Projeção ortogonal
 - Projeção perspectiva simétrica (e oblíqua)
- A biblioteca **OpenGL Utility (GLU)** inclui funções para
 - Especificar os parâmetros de visão (posição, direção e orientação da câmera)
 - Definir a transformação de projeção perspectiva simétrica

Função de projeção ortogonal OpenGL

- Para se definir as matrizes de projeção é necessário especificar que se está trabalhando com a matriz PROJECTION

```
glMatrixMode(GL_PROJECTION);
```

- Para se definir uma projeção ortogonal, a seguinte função é chamada

```
glOrtho(GLdouble xmin, GLdouble xmax, GLdouble  
        ymin, GLdouble ymax, GLdouble dnear, GLdouble  
        dfar);
```

- Os 4 primeiros parâmetros definem as coordenadas da janela de recorte e os 2 últimos as distâncias para os planos de recorte near/far

Funções OpenGL para viewing 3D

- O plano de visão é sempre coincidente com o plano de recorte near
- Os parâmetros dnear e dfar denotam distâncias na direção negativa de z_{view} do sistema de visão
 - Por exemplo, se dfar = 55, o plano de recorte far estará na posição $z_{\text{far}} = -55$
 - No OpenGL, quaisquer valores podem ser atribuídos, normalmente dnear < dfar
- Por padrão, a OpenGL usa os seguintes valores para essa função

```
glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
```

Função de projeção ortogonal OpenGL - exemplo 1

```
#include <windows.h>
#include <GL/freeglut.h>

int init(){
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f); //define a cor de fundo
    glEnable(GL_DEPTH_TEST); //habilita o teste de profundidade
    glMatrixMode(GL_PROJECTION); //define que a matrix é a de projeção
    glLoadIdentity(); //carrega a matrix de identidade
    glOrtho(-2.0, 2.0, -2.0, 2.0, -2.0, 2.0); //define uma projeção ortogonal
}

void display(){
    //limpa o buffer
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    //define que a matrix é a de modelo
    glMatrixMode(GL_MODELVIEW);

    //desenha um cubo aramado vermelho com aresta 1.0 de comprimento,
    //centrado na origem, indo de -0.5 a +0.5 em x, y e z
    glColor3f(1.0f, 0.0f, 0.0f);
    glutWireCube(1.0f);

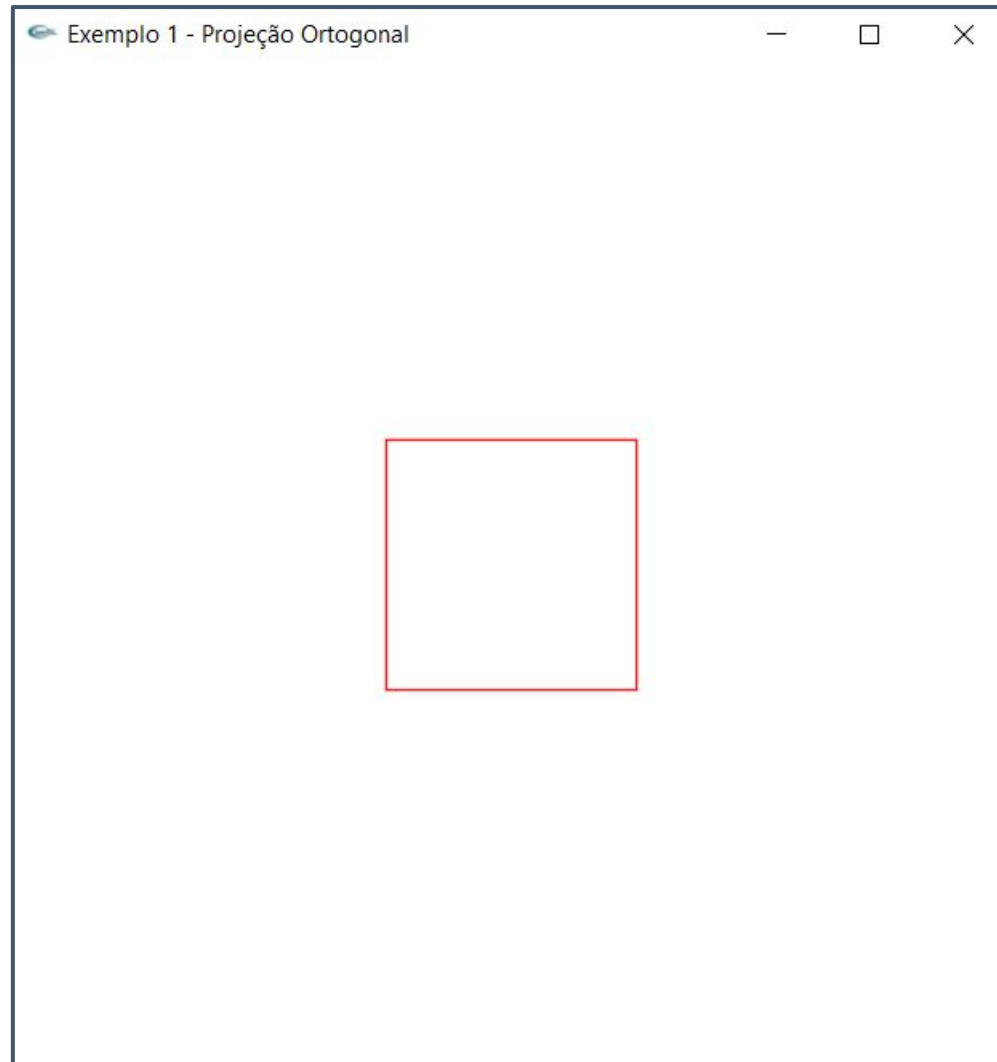
    //força o desenho das primitivas
    glFlush();
}
```

Função de projeção ortogonal OpenGL - exemplo 1

```
int main(int argc, char** argv){
    glutInit(&argc, argv);           //inicializa o GLUT
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH); //configura o modo de display
    glutInitWindowPosition(200, 0);  //seta a posição inicial da janela
    glutInitWindowSize(500, 500);    //configura a largura e altura da janela de exibição
    glutCreateWindow("Exemplo 1 - Projeção Ortogonal"); //cria a janela de exibição

    init();                          //executa função de inicialização
    glutDisplayFunc(display);
    glutMainLoop();                  //mostre tudo e espere
    return 0;
}
```

Função de projeção ortogonal OpenGL - exemplo 1



Função de transformação de visão OpenGL

- Os parâmetros de visão formam uma matriz que é concatenada com a matriz MODELVIEW corrente, então inicia-se especificando

```
glMatrixMode (GL_MODELVIEW) ;
```


Função de transformação de visão OpenGL

- Para especificar os **parâmetros de visão** usamos a função

```
gluLookAt(x0, y0, z0, xref, yref, zref, Vx, Vy,  
          Vz);
```

- Essa função define
 - A origem do sistema de visão $P_0 = (x_0, y_0, z_0)$ no sistema de coordenadas de mundo (a localização da câmera)
 - A posição de referência $P_{ref} = (x_{ref}, y_{ref}, z_{ref})$ (para onde a câmera aponta)
 - O vetor view-up $V = (V_x, V_y, V_z)$

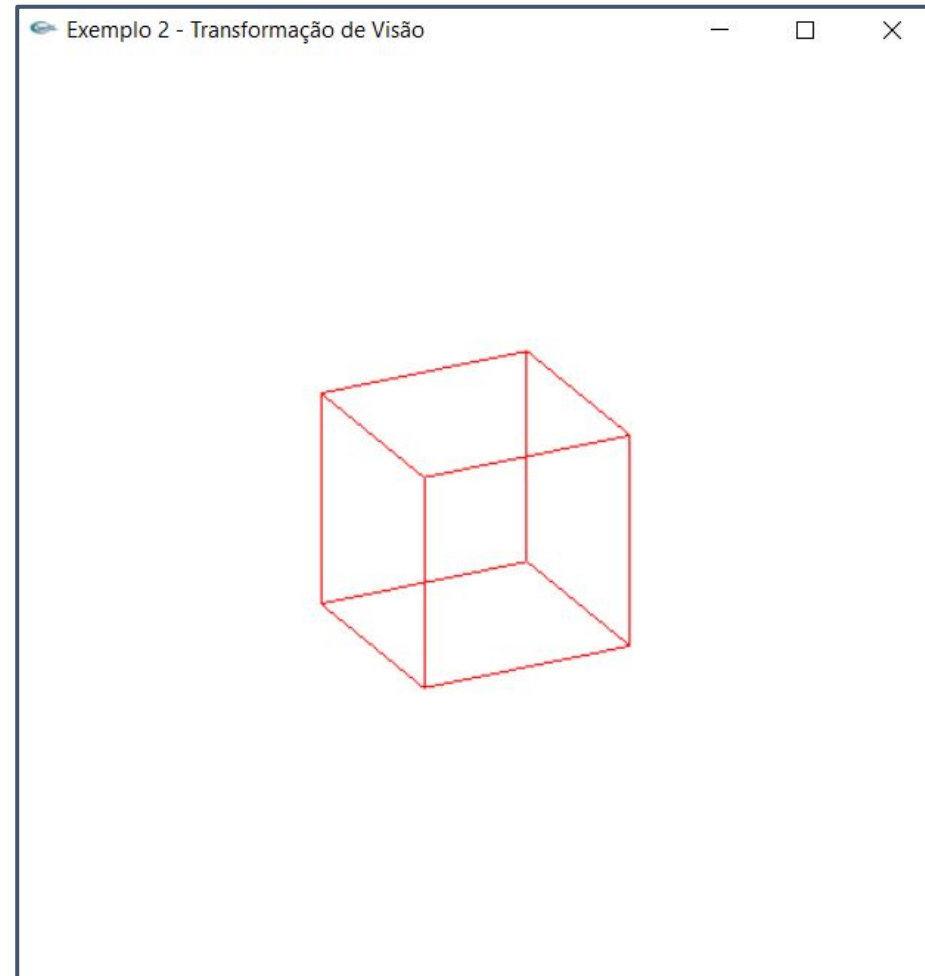
Função de transformação de visão OpenGL

- A direção positiva do eixo z_{view} do sistema de coordenadas de visão está na direção $N = P_0 - P_{\text{ref}}$
 - A direção de visão está na direção negativa do eixo z_{view}
- Os parâmetros especificados usando a função `gluLookAt(...)` são usados para compor a matriz $M_{WC,VC}$
 - Matriz que alinha o sistema de visão com o sistema de coordenadas de mundo
- Por padrão os parâmetros de visão da `gluLookAt(...)` são
 - $P_0 = (0, 0, 0)$
 - $P_{\text{ref}} = (0, 0, -1)$
 - $V = (0, 1, 0)$

Função de transformação de visão OpenGL - exemplo 2

```
int init() {  
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f); //define a cor de fundo  
    glEnable(GL_DEPTH_TEST); //habilita o teste de profundidade  
  
    glMatrixMode(GL_MODELVIEW); //define que a matrix é a model view  
    glLoadIdentity(); //carrega a matrix de identidade  
    gluLookAt(0.8, 0.5, 0.5, //posição da câmera  
              0.0, 0.0, 0.0, //para onde a câmera aponta  
              0.0, 1.0, 0.0); //vetor view-up  
  
    glMatrixMode(GL_PROJECTION); //define que a matrix é a de projeção  
    glLoadIdentity(); //carrega a matrix de identidade  
    glOrtho(-2.0, 2.0, -2.0, 2.0, -2.0, 2.0); //define uma projeção ortogonal  
}
```

Função de transformação de visão OpenGL - exemplo 2



Função para projeção perspectiva simétrica OpenGL

- Para se criar um volume de visão frustum simétrico sobre a direção de visão (direção negativa do eixo z_{view}) usamos

```
gluPerspective(theta, aspect, dnear, dfar);
```

- Os parâmetros theta e aspect definem o tamanho e posição da janela de recorte
 - $0^\circ \leq \text{theta} \leq 180^\circ$ define o ângulo do campo de visão/altura
 - aspect é o valor da razão de aspecto width/height da janela de recorte
- Os parâmetros dnear e dfar especificam as distâncias do ponto de visão (origem do sistema de coordenadas) para os planos de recorte near e far

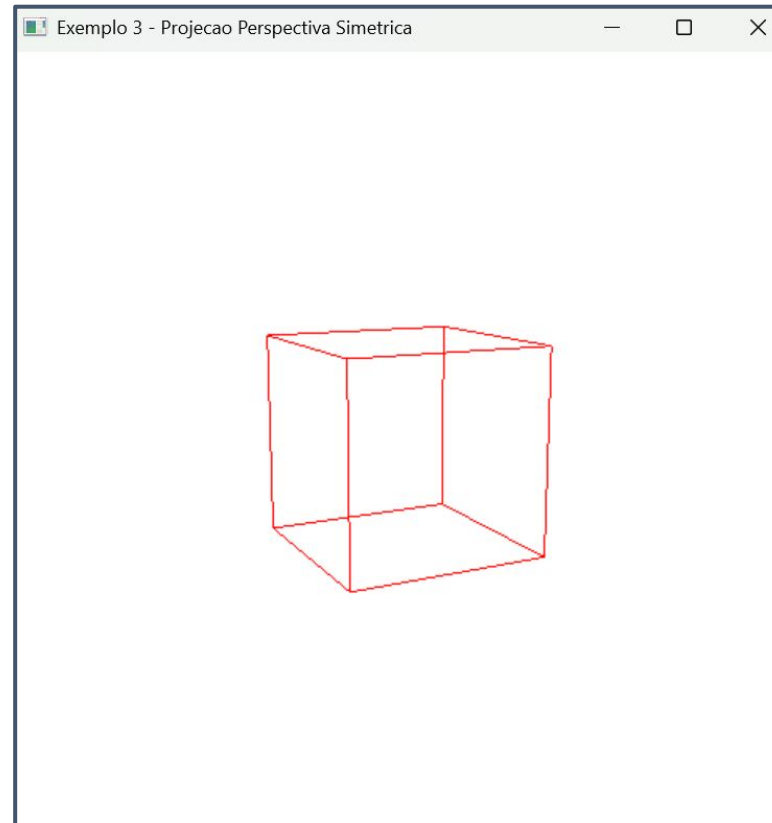
Função para projeção perspectiva simétrica OpenGL

- O ponto de visão (posição da câmera) é a origem do sistema de coordenadas de visão
- O plano de recorte near coincide com o plano de visão
- Os planos de recorte near/far podem estar ao longo da direção negativa de z_{view} e não podem estar atrás da posição de visão/câmera
- Os valores devem ser $d_{\text{near}} < d_{\text{far}}$

Função para projeção perspectiva simétrica

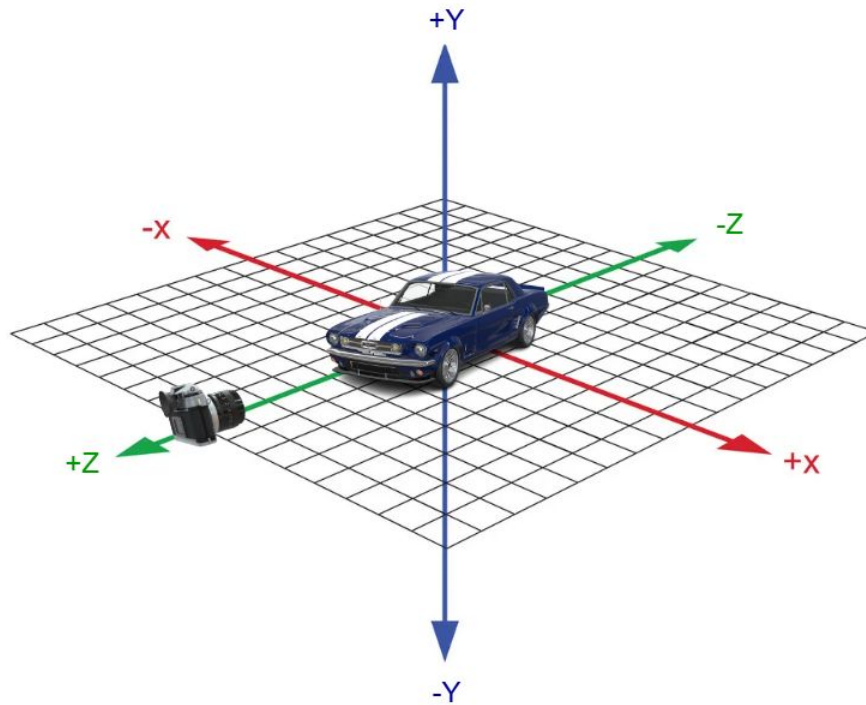
OpenGL - exemplo 3

- **Atividade:** modifique o código anterior para visualizar o cubo na projeção perspectiva



Exemplos alterando os parâmetros da gluLookAt

```
gluLookAt(0.0, 0.0, 1.0, // posição da câmera  
          0.0, 0.0, 0.0, // para onde a câmera aponta  
          0.0, 1.0, 0.0); // vetor view-up
```

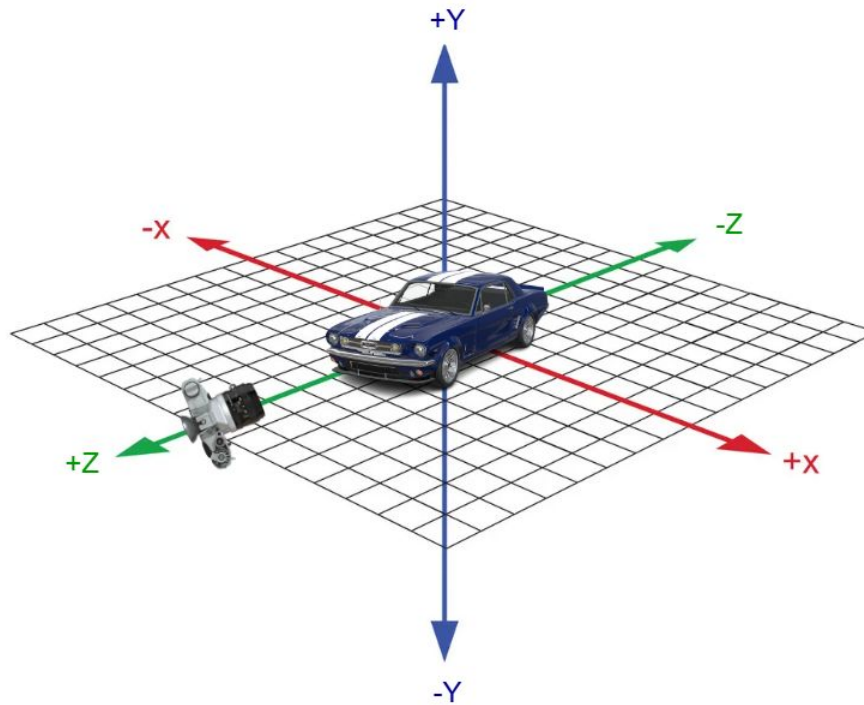


O que a câmera captura



Exemplos alterando os parâmetros da gluLookAt

```
gluLookAt(0.0, 0.0, 1.0, // posição da câmera  
          0.0, 0.0, 0.0, // para onde a câmera aponta  
          1.0, 0.0, 0.0); // vetor view-up
```

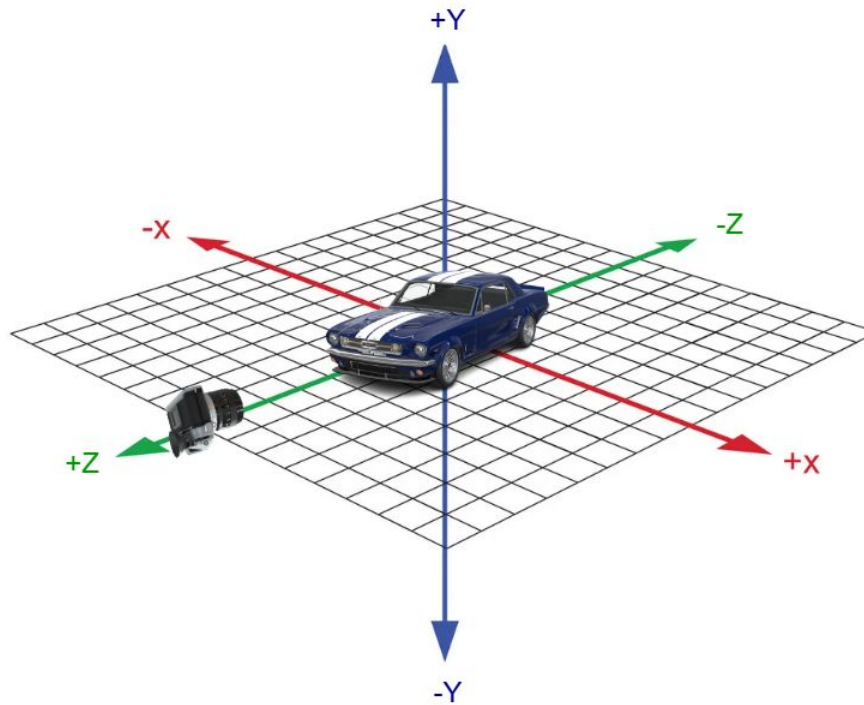


O que a câmera captura



Exemplos alterando os parâmetros da gluLookAt

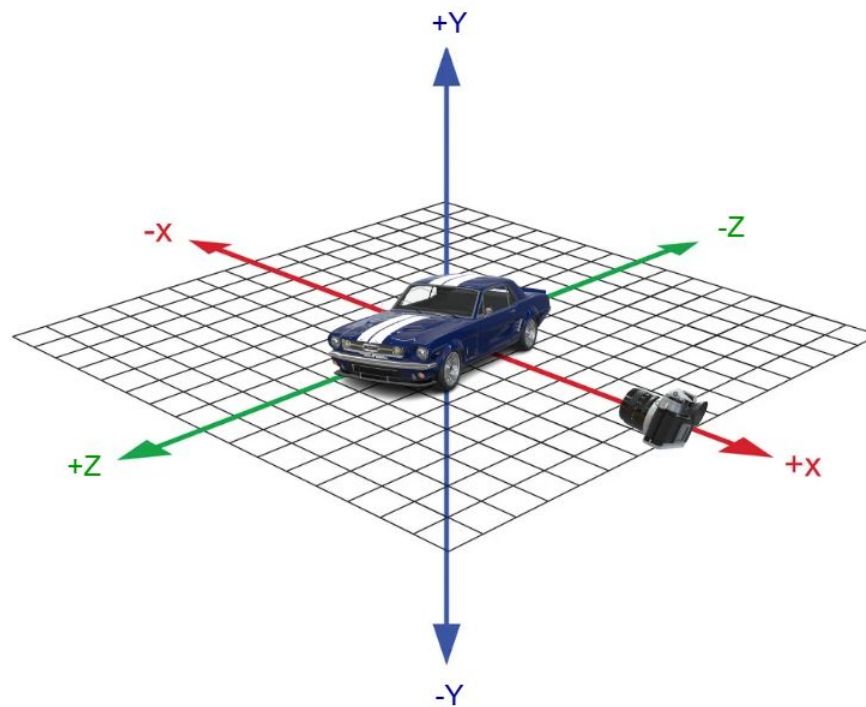
```
gluLookAt(0.0, 0.0, 1.0, // posição da câmera  
          0.0, 0.0, 0.0, // para onde a câmera aponta  
          0.0, -1.0, 0.0); // vetor view-up
```



O que a câmera captura



Exemplos alterando os parâmetros da gluLookAt

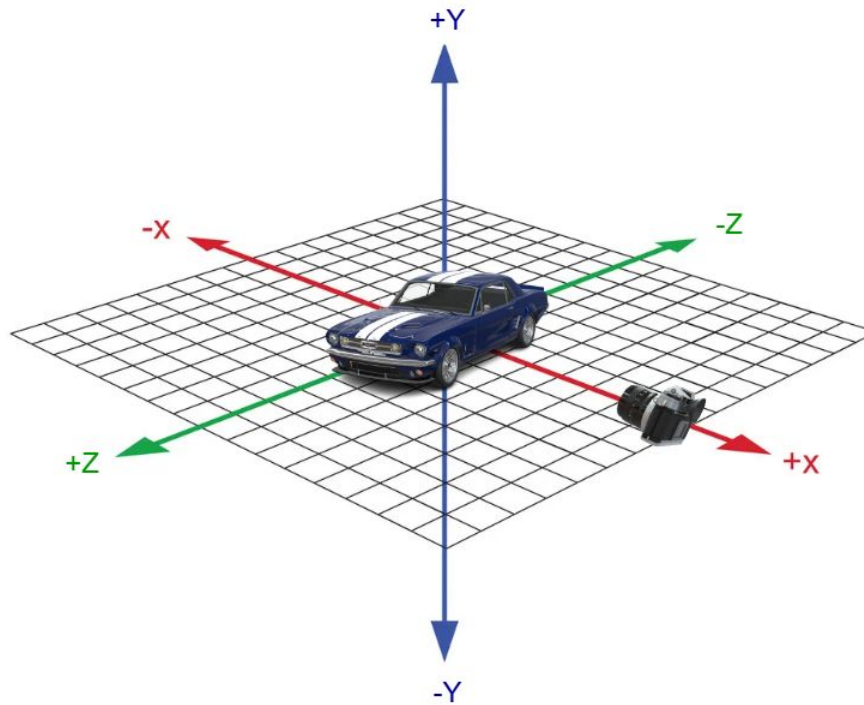


O que a câmera captura



Exemplos alterando os parâmetros da gluLookAt

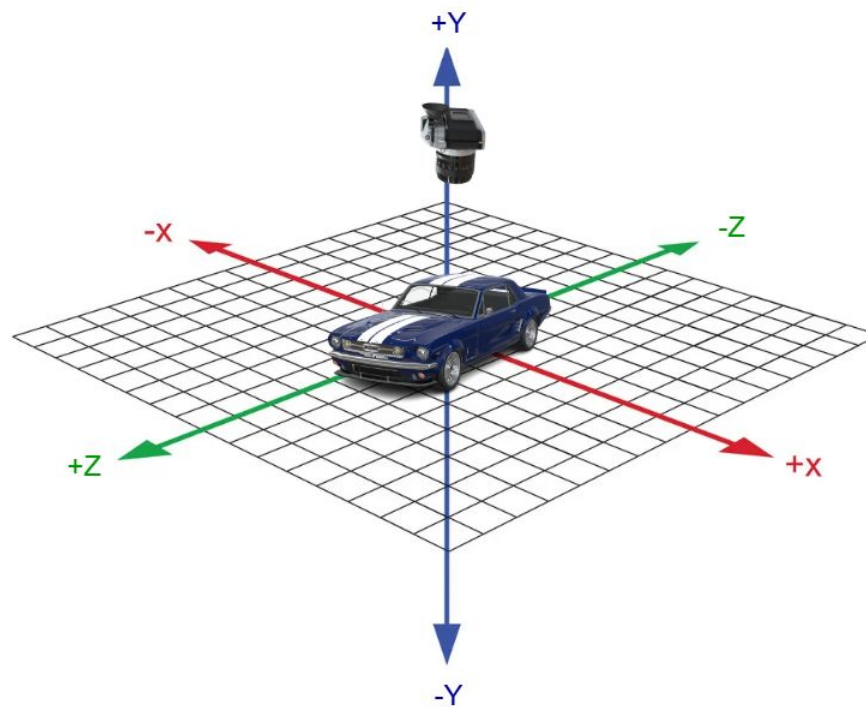
```
gluLookAt(1.0, 0.0, 0.0, // posição da câmera  
          0.0, 0.0, 0.0, // para onde a câmera aponta  
          0.0, 1.0, 0.0); // vetor view-up
```



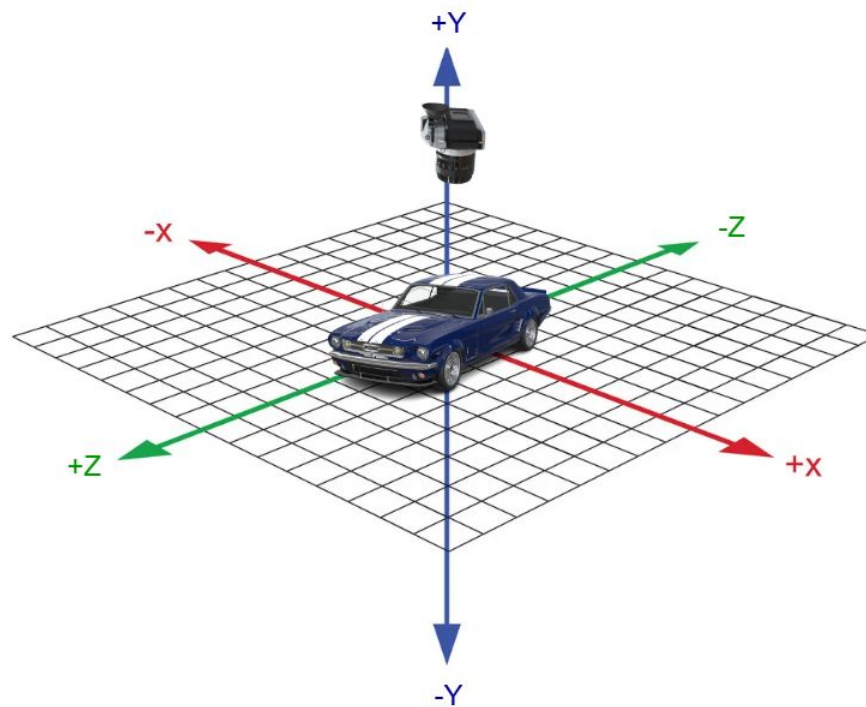
O que a câmera captura



Exemplos alterando os parâmetros da gluLookAt



Exemplos alterando os parâmetros da gluLookAt

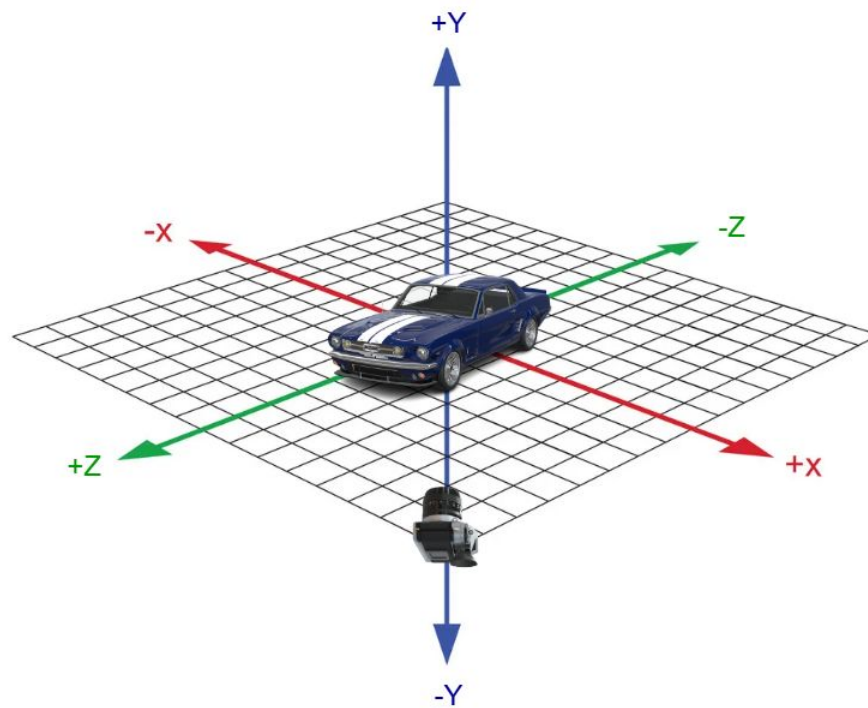


```
gluLookAt(0.0, 1.0, 0.0, // posição da câmera  
          0.0, 0.0, 0.0, // para onde a câmera aponta  
          0.0, 0.0, 1.0); // vetor view-up
```

O que a câmera captura

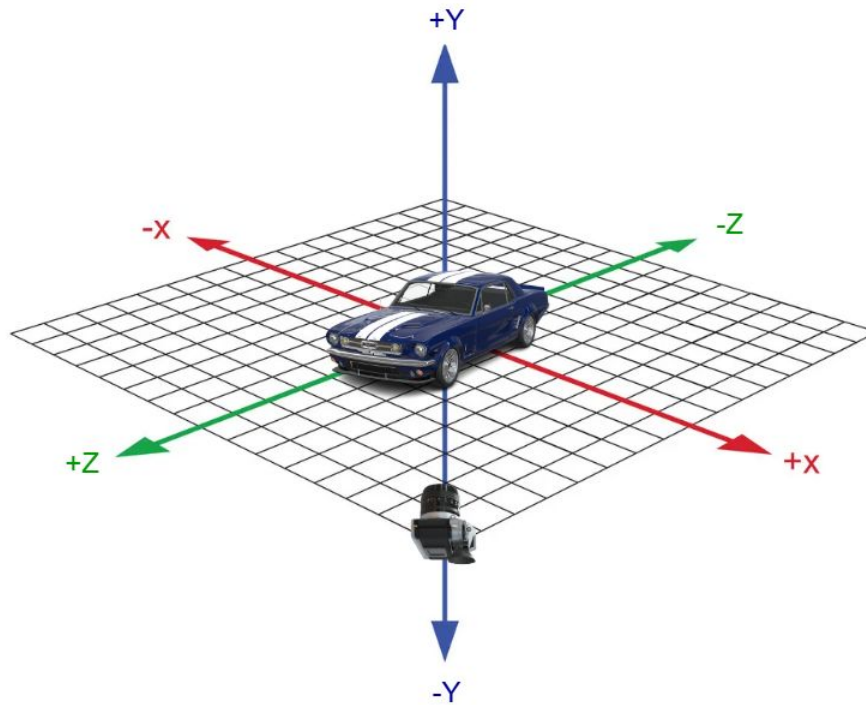


Exemplos alterando os parâmetros da gluLookAt



Exemplos alterando os parâmetros da gluLookAt

```
gluLookAt(0.0, -1.0, 0.0, // posição da câmera  
          0.0, 0.0, 0.0, // para onde a câmera aponta  
          0.0, 0.0, -1.0); // vetor view-up
```



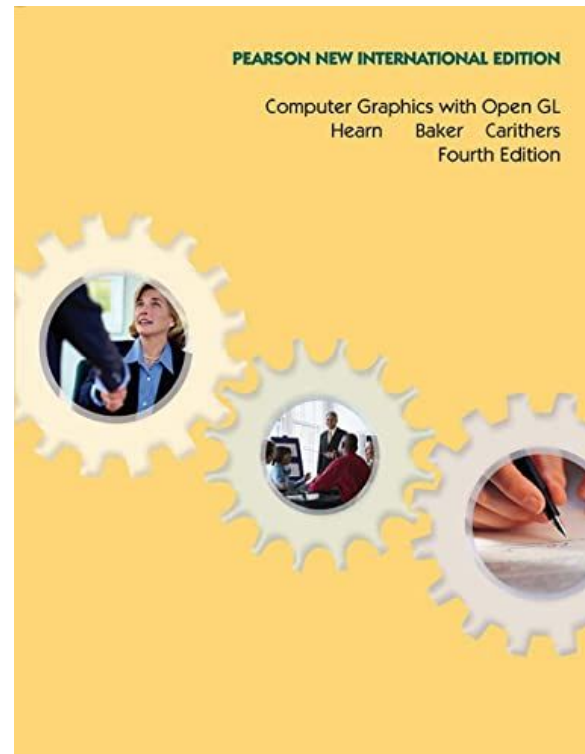
O que a câmera captura



Referências



AZEVEDO, Eduardo; CONCI, Aura.
Computação gráfica: teoria e prática.
Elsevier, 2003.

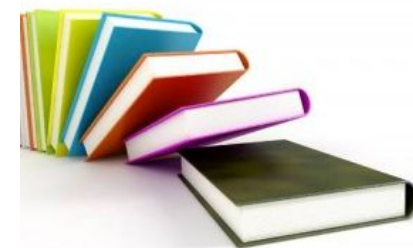


HEARN, DONALD. **Computer graphics with OpenGL.** Pearson Education Limited, 2014.
5º Edição.



PICHETTI, R. F; et al. **Computação Gráfica e Processamento de Imagens.**
Porto Alegre: SAGAH, 2022.

Referências



- ALENCAR, Aretha. **Notas de aula da disciplina Computação Gráfica da Universidade Tecnológica Federal do Paraná.** 2021.
- PAIVA, Anselmo. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2018.
- QUINTANILHA, Darlan. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2022.
- SILVA, Giovanni. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2019.